

OJTFlow

A Governed Multi-Agent Platform for Medical and Healthcare Data Collaboration

Foundation Models, RAG, MCP Tools, and Human-in-the-Loop Control

Proposal Thesis

OJTFlow turns JSON, YAML, CSV, and healthcare structured-data workflows into explainable, auditable, agent-assisted data operations. The system uses foundation models for intent, reasoning, and explanation, while deterministic tools, retrieval evidence, validation reports, and clinician-facing transparency controls preserve correctness and interpretability.

Document type: Comprehensive project proposal and implementation plan

Prepared for: HuReDee-IoYou AI OJT Program / Project Team Review

Prepared by: Nguyen Tran Nhat Trung; Nguyen Tran Minh Khoi; Tran Ha Son

Version: Research-backed LaTeX proposal v1.0

Date: May 13, 2026

Proposed MVP duration: 8 weeks



Abstract

OJTFlow is a proposed medical and healthcare AI platform for governed structured-data collaboration, multimodal medical data processing, natural language interaction, and evidence-grounded explanation. The system is designed to help teams transform, validate, retrieve, explain, and audit healthcare data across JSON, YAML, CSV, FHIR, Bulk FHIR, OMOP-aligned tables, scanned clinical documents, DICOM images, and medical video workflows. It combines deterministic data engineering with classical machine learning, deep learning, foundation AI, advanced retrieval, Graph-NER, OCR, medical image segmentation, object detection/tracking, self-supervised representation learning, and clinician-in-the-loop governance.

The central thesis is that healthcare AI systems should not rely on a foundation model alone. OJTFlow uses foundation models for intent understanding, planning, summarization, visual prompting, and explanation, while deterministic validators, schema tools, FHIR/OMOP/JP Core constraints, retrieval evidence, DICOM/OCR metadata, audit logs, and human review preserve correctness and accountability. The proposal extends conventional RAG with hybrid retrieval, HyDE, reranking, hierarchical retrieval, GraphRAG, Graph-NER, optional GNN reranking, and self-supervised embedding adaptation. It also includes healthcare-grade explainability: supported-claim mapping, source provenance, uncertainty, abstention, limitations, image overlays, OCR bounding boxes, segmentation masks, tracking traces, and clinical review checkpoints.

The project is positioned from both research and industrial perspectives for the Japanese healthcare market. As research, it explores advanced retrieval, medical entity graphs, representation learning, explainable foundation AI, promptable medical segmentation, clinical document intelligence, and medical imaging evaluation. As industry architecture, it specifies a deployable GCP-first MLOps and CI/CD stack using GitHub Actions, Workload Identity Federation, Artifact Registry, Cloud Deploy or Argo CD, GKE, Cloud Run, Terraform, Helm, MLflow, Kubeflow Pipelines, Feast, KServe, vLLM, Ray Serve, Qdrant or Milvus, OpenSearch, graph databases, Cloud Healthcare API, Document AI, OpenTelemetry, Cloud Monitoring, Prometheus, Grafana, and Evidently. Medical constraints are treated as first-class requirements, including APPI/Japan privacy expectations, MHLW medical information security guidance, PMDA/SaMD boundary awareness, PHI protection, de-identification, Good Machine Learning Practice, change control, bias monitoring, external validation, auditability, and reporting alignment with FUTURE-AI, TRIPOD+AI, CONSORT-AI, DECIDE-AI, and CLAIM.

Proposal Snapshot

Project	OJTFlow: Medical and healthcare AI workflow platform.
Core purpose	Convert, validate, retrieve, explain, and audit structured healthcare data, clinical documents, DICOM imaging, and medical video with natural language control.
AI approach	Classical ML + deep learning + foundation AI + Graph-NER + OCR + SAM/MedSAM-style segmentation + detection/tracking + advanced RAG + self-supervised representation learning.
Healthcare focus	Japan-ready medical DX, JP Core/FHIR, Bulk FHIR, OMOP, scanned forms, DICOM, Synthea, MIMIC-IV, eICU, n2c2/i2b2, MedQA, PubMedQA, MIMIC-CXR, CheXpert, public segmentation/detection datasets.
Safety principle	No autonomous diagnosis or treatment in the MVP; all medically consequential actions require human or clinician review.
Industrial stack	FastAPI, LangGraph, MCP, PostgreSQL, Qdrant/Milvus, OpenSearch, graph DB, GCP Healthcare API, Document AI, MLflow, Kubeflow, Feast, KServe, vLLM, GKE/Cloud Run, GitHub Actions, Cloud Deploy/Argo CD.
Expected output	A deployable prototype, evaluation suite, audit trail, medical explanation report, and MLOps-ready architecture.

Contents

Abstract	i
1 Project Statement	1
1.1 Project Title	1
1.2 Statement of Intent	1
1.3 Core Objectives	1
1.4 Team Vision	1
2 Executive Summary	2
2.1 MVP Outcome	2
2.2 Why This Proposal Is Strong	3
3 Research Basis and Design Rationale	3
3.1 Evidence Matrix	3
3.2 Design Conclusion	5
4 Problem Statement and Opportunity	5
4.1 The Practical Problem	5
4.2 Opportunity for the OJT Project	5
4.3 Target Users	6
5 Proposed Solution and System Architecture	6
5.1 Product Concept	6
5.2 Primary Capabilities	6
5.3 Non-Goals for the MVP	7
5.4 Architecture Overview	7
5.5 Layered Architecture	7
6 Core Components and Agent Contracts	8
6.1 Agent Roles	8
6.2 Shared Workflow State	9
6.3 Agent Response Contract	9
6.4 Workflow Example	10
7 RAG, MCP, and Foundation Model Strategy	10
7.1 RAG Design	10
7.1.1 Knowledge Sources	10
7.1.2 Retrieval Pipeline	10

7.1.3	RAG Quality Controls	11
7.2	Advanced Retrieval Strategy	11
7.2.1	Recommended Retrieval Ladder	11
7.2.2	OJTFlow Knowledge Graph Design	13
7.2.3	Graph-NER for Medical Entity Recognition and Schema Linking	13
7.2.4	Safety Rules for Advanced Retrieval	14
7.3	Self-Supervised Representation Learning Strategy	15
7.3.1	Research-Informed Methods	15
7.3.2	Project-Specific Pretext Tasks	16
7.3.3	Recommended Implementation Path	16
7.3.4	Foundation Model Boundary	17
7.4	MCP Integration Design	17
7.4.1	Example MCP Tool Contracts	18
7.5	Foundation Model Strategy	18
7.6	Recommended MVP Position	18
8	Security, Governance, and Human-in-the-Loop Controls	18
8.1	Key Risks and Mitigations	19
8.2	Human Review Triggers	19
8.3	Audit Requirements	20
8.4	Governance Principle	20
9	Medical and Healthcare Explainability Strategy	20
9.1	Explainability Position	20
9.2	SOTA Explainability Methods to Apply	21
9.3	Clinical Explanation Output Contract	22
9.4	Healthcare Safety and Governance Rules	23
9.5	Medical Evaluation Plan	23
10	Datasets, AI Methods, Database Stack, CI/CD, and MLOps	23
10.1	Research and Industrial Positioning	24
10.2	Healthcare Dataset Strategy	24
10.3	Dataset-to-Purpose Mapping	26
10.4	Medical Constraints and Requirements	26
10.5	Clinical AI Acceptance Gates	27
10.6	AI Method Portfolio: ML, Deep Learning, and Foundation AI	28
10.7	Advanced Data and Retrieval Storage Stack	29
10.8	Full CI/CD, GitOps, and MLOps Pipeline	30
10.9	GCP-First Ops and MLOps Stack	31

10.9.1 GCP Reference Deployment	33
10.9.2 GitHub Actions + GCP CI/CD and MLOps Blueprint	33
10.10 Monitorability and Observability by Design	34
10.10.1 Core SLOs and Alerts	35
10.10.2 Runbooks and Incident Response	35
10.11 MLOps Lifecycle for Medical AI	36
10.12 Recommended Advanced Stack	37
11 Multimodal Medical AI and Japan Market Strategy	37
11.1 Japan-Market Rationale	37
11.2 Japan-Specific Compliance and Adoption Controls	38
11.3 Multimodal Module Portfolio	38
11.4 Medical Vision Research Stack	39
11.5 How Multimodal AI Connects to RAG, Graph-NER, and SSL	40
11.6 GCP-First Industrial Architecture for Multimodal Healthcare	40
11.7 Realistic 12-Week Prototype Scope	41
12 12-Week Implementation Plan and Personnel Work Breakdown	41
12.1 Planning Assumptions	41
12.2 Member Slot Ownership	42
12.3 12-Week Master Schedule	42
12.4 Detailed Weekly Work Plan by Member Slot	43
12.5 Phase Gates and Promotion Criteria	45
13 Technical Stack and API Design	46
13.1 Recommended Stack	46
13.2 API Endpoints	47
13.3 Example Workflow Request	47
13.4 Example Workflow Response	47
14 Evaluation Plan and Implementation Roadmap	47
14.1 Evaluation Dimensions	48
14.2 Test Dataset Plan	49
14.3 Roadmap Summary	49
14.4 MVP Backlog	50
15 Expected Outcomes, Risks, and Future Extensions	51
15.1 Expected Outcomes	51
15.2 Project Risks	52
15.3 Future Extensions	52

15.4 Final Positioning	53
16 References	53

1 Project Statement

1.1 Project Title

Multi-Agent AI System for Structured Data Collaboration and Natural Language Interaction

1.2 Statement of Intent

This project proposes the design and development of OJTFlow, a lightweight but credible multi-agent AI platform that helps people and software systems collaborate around structured data. The system accepts JSON, YAML, and CSV inputs, interprets natural language instructions, coordinates specialized agents, invokes validated data tools through the Model Context Protocol (MCP), retrieves trusted schema and transformation knowledge through retrieval-augmented generation (RAG), and returns correct outputs with clear explanations.

The goal is not to build another file converter. The goal is to build an intelligent structured-data operations layer: a system that can convert, validate, explain, audit, and ask for human approval when a transformation could change meaning. This direction fits the HuReDee-IOYou AI OJT program because it demonstrates practical backend engineering, model integration, agent orchestration, protocol design, deployment readiness, and product thinking in one cohesive prototype.

For the medical and healthcare application track, OJTFlow should be treated as clinical data support infrastructure, not an autonomous diagnosis system. It should help transform, validate, retrieve, summarize, and explain healthcare data while preserving provenance, uncertainty, limitations, human accountability, and clinician review.

1.3 Core Objectives

- Design a modular multi-agent architecture for structured-data workflows.
- Support reliable conversion among JSON, YAML, and CSV while preserving schema intent.
- Enable natural language control for conversion, validation, cleaning, and explanation tasks.
- Use RAG to ground decisions in approved schemas, transformation examples, and data dictionaries.
- Apply advanced retrieval and self-supervised representation learning to improve schema matching, medical terminology alignment, and evidence retrieval.
- Expose deterministic data tools through MCP servers so agents use standard, auditable interfaces.
- Add clinician/human-in-the-loop checkpoints for ambiguous, destructive, sensitive, or medically consequential operations.
- Provide healthcare-grade explainability: evidence traces, uncertainty, source provenance, data-quality warnings, and interpretable user-facing reports.
- Deliver a deployable API-driven prototype with documentation, tests, evaluation metrics, and a demonstration scenario.

1.4 Team Vision

The team vision is to bridge two worlds that usually stay separated: strict machine-readable data and natural human collaboration. OJTFlow should let a non-technical user ask, “Clean this CSV and explain the anomalies,” while the system behaves like disciplined software: parsing exactly, validating against explicit rules, preserving provenance, and refusing risky actions until a person approves them.

2 Executive Summary

OJTFlow is a proposed multi-agent, multimodal medical AI platform for governed healthcare data collaboration. Users can provide structured files, FHIR/OMOP-like records, scanned clinical documents, DICOM images, or medical video and issue instructions such as “validate this lab CSV against the approved schema,” “extract fields from this referral form,” “map this bundle to the hospital data model,” or “segment and explain the region of interest for review.” The platform routes each request through a governed workflow where foundation models handle intent, planning, visual prompting, and explanation, while deterministic tools perform parsing, validation, conversion, OCR/layout extraction, DICOM metadata handling, persistence, and audit.

The proposal combines four architecture patterns that are now central to production-grade AI applications:

- **Foundation model interface:** interprets natural language, creates workflow plans, selects tools, summarizes validation findings, and produces human-friendly explanations.
- **Retrieval-augmented generation:** retrieves trusted schemas, field definitions, transformation policies, historical examples, and error-handling guidance before the model explains or plans.
- **Model Context Protocol integration:** exposes conversion, validation, retrieval, review, and audit capabilities as discoverable MCP tools, resources, and prompts.
- **Multi-agent orchestration:** divides work across specialized agents for input parsing, schema inference, validation, transformation, retrieval, explanation, safety, and human review.
- **Multimodal medical AI:** adds OCR, document layout understanding, DICOM ingestion, promptable segmentation, object detection/tracking, and image/video evidence overlays for clinician review.

For the advanced research track, the proposal also includes SOTA retrieval, representation-learning, and medical vision modules: HyDE query expansion, reranking, hierarchical/RAPTOR-style retrieval, GraphRAG, lightweight graph retrieval, optional GNN-based graph reranking, contrastive embedding adaptation, denoising representation learning, BYOL-style no-negative learning, tabular SSL for healthcare tables, medical OCR representation learning, SAM/MedSAM-style promptable segmentation, SAM2/MedSAM-2-style image/video tracking, nnU-Net baselines, and medical object detection. These methods are framed as measurable modules with promotion gates, so the project can be ambitious without becoming scientifically careless.

The central engineering principle is separation of judgment from execution. The foundation model can reason, ask questions, and explain; it should not be trusted as the only mechanism for producing structured data. Exact transformations should be performed by code and verified by validators. This design directly addresses common LLM failure modes: hallucinated fields, invalid JSON, unsafe tool calls, prompt injection, sensitive data leakage, and silent schema drift.

For medical and healthcare use, the system adds a second principle: separation of explanation from unsupported rationalization. OJTFlow should explain through evidence traces, validation outputs, source provenance, OCR bounding boxes, segmentation masks, detection boxes, tracking timelines, uncertainty, and clinician review, not through unverified model self-explanation alone. Because Japan is the target market, the full proposal also treats JP Core/FHIR alignment, MHLW Medical DX direction, APPI-sensitive medical data handling, PMDA/SaMD boundary awareness, and Japanese enterprise adoption style as design inputs rather than afterthoughts.

2.1 MVP Outcome

The MVP will deliver a deployable API and simple user interface that demonstrate an end-to-end workflow:

1. User uploads a messy CSV and asks for cleaning, JSON conversion, and explanation.
2. Parser and schema agents profile the input.
3. RAG retrieves an approved schema and cleaning rules.
4. Validation identifies missing values, malformed rows, and type inconsistencies.
5. Human review is triggered for semantic changes such as dropping rows or filling missing values.
6. Transformation tools convert the approved data into JSON.
7. The system returns output, validation report, explanation, provenance, and audit trail.

2.2 Why This Proposal Is Strong

- It is ambitious enough to demonstrate modern AI engineering, but scoped enough to ship in an OJT timeline.
- It uses current AI infrastructure ideas without overpromising full autonomy or custom model training.
- It produces visible value for both technical and non-technical users.
- It includes evaluation, safety, deployment, and human oversight from the beginning.
- It can grow into a reusable middleware layer for APIs, documents, dashboards, internal tools, and enterprise workflows.

3 Research Basis and Design Rationale

The proposal is grounded in current research and official documentation for foundation models, RAG, MCP, multi-agent systems, human oversight, and AI security. The research points toward one conclusion: the most credible architecture is not a single chatbot, but a tool-grounded agent workflow with retrieval, validation, consent, and auditability.

3.1 Evidence Matrix

Area	Research or documentation finding	Design implication
Foundation models	Stanford CRFM defines foundation models as models trained on broad data, generally with self-supervision at scale, that can be adapted to many downstream tasks [1].	Use a foundation model for language understanding, planning, explanation, and flexible reasoning. Do not rely on it as the only source of truth for deterministic data transformation.
RAG	Lewis et al. describe RAG as combining parametric model memory with non-parametric memory, addressing provenance and knowledge updating limitations in model-only systems [2].	Ground explanations and transformation plans in retrievable project knowledge: schemas, examples, documentation, and approved rules.

Advanced retrieval	HyDE improves zero-shot dense retrieval through hypothetical document embeddings [3]; RAPTOR retrieves over a recursive summary tree [4]; GraphRAG builds entity graphs and community summaries for local and global sensemaking [5].	Treat retrieval as a modular stack: hybrid first-stage retrieval, reranking, hierarchical retrieval, entity-aware graph retrieval, and optional graph-neural ranking for hard multi-hop schema questions.
Self-supervised representation learning	SimCSE, Contriever, E5, and contrastive text-code embedding research show that contrastive or weakly supervised pretraining can produce strong retrieval embeddings [21, 22, 23, 24]. BYOL-style learning shows that useful representations can also be learned without explicit negative pairs [26].	Adapt the embedding and schema-representation layer using unlabeled project artifacts: schemas, fields, examples, conversions, validation errors, and data dictionaries. Keep foundation-model pretraining out of MVP scope.
Explainability in medical AI	FDA/Health Canada/MHRA transparency principles for machine-learning-enabled medical devices emphasize intended users, workflow fit, performance, risks, limitations, logic where available, and human-centered communication [11]. DECIDE-AI emphasizes early clinical evaluation, safety, human factors, and transparent reporting for AI decision support [12].	Design explanations as clinical safety artifacts: retrieved evidence, validation trace, uncertainty, abstention, limitations, user-facing rationale, and audit logs. Avoid treating model self-explanations or attention maps as sufficient proof.
MCP	MCP is a JSON-RPC based client-server protocol where servers expose tools for actions, resources for context, and prompts for reusable interaction templates [31, 32].	Wrap conversion, validation, retrieval, review, and audit functions as MCP servers with well-defined input schemas and access controls.
MCP operations	MCP supports local stdio and remote Streamable HTTP transports, capability discovery, notifications, progress tracking, and user elicitation patterns [31].	Start locally with stdio during development; design the server layer so it can move to remote Streamable HTTP deployment later.
Multi-agent systems	AutoGen research shows that LLM applications can be built from customizable conversable agents that combine LLMs, tools, and human participation [34].	Use specialized agents rather than one monolithic prompt, with explicit roles and state transitions.
Agent workflows	LangGraph emphasizes durable execution, streaming, persistence, and human-in-the-loop support for agent orchestration [35].	Use graph-based orchestration for workflows that can pause, resume, show progress, and preserve state for audit.
LLM security	OWASP's LLM risk guidance identifies prompt injection, sensitive information disclosure, supply chain risk, improper output handling, excessive agency, vector/embedding weaknesses, misinformation, and unbounded consumption [36].	Treat user data and retrieved text as untrusted, validate model outputs, limit tool permissions, and require review for risky actions.

Risk management	NIST AI 600-1 provides a generative AI profile for identifying, measuring, managing, and governing AI risks [37].	Include evaluation, documentation, audit evidence, human accountability, and monitoring in the project plan.
-----------------	---	--

3.2 Design Conclusion

OJTFlow should be positioned as **governed structured-data AI middleware**. Its value comes from combining language understanding with exact tools, not from replacing software engineering with free-form generation. The architecture should therefore enforce five rules:

1. Use the model to interpret intent, not to silently mutate data.
2. Use deterministic parsers and validators for all data transformations.
3. Use RAG only from trusted, metadata-scoped knowledge sources.
4. Use MCP to make tools discoverable, typed, and auditable.
5. Use human review whenever a transformation can change meaning, reveal sensitive data, or call an external system.

4 Problem Statement and Opportunity

4.1 The Practical Problem

Modern teams exchange structured data through APIs, spreadsheets, configuration files, dashboards, ETL jobs, notebooks, and AI tools. JSON, YAML, and CSV are everywhere, but everyday workflows remain surprisingly fragile:

- A user may not know whether a file is valid, what schema it follows, or how to safely convert it.
- Developers often need repeatable conversion logic, not a one-off answer from a chatbot.
- Non-technical users need explanations of fields, rows, anomalies, and business meaning.
- LLM-only systems can produce plausible but invalid JSON, invent fields, or ignore schema constraints.
- Real workflows require logs, versioned schemas, approvals, access control, and traceability.

The gap is not simply “format conversion.” The gap is collaborative structured-data operations: people need a system that can understand an instruction, inspect the data, retrieve the right schema context, perform the transformation exactly, explain the result, and prove what happened.

4.2 Opportunity for the OJT Project

OJTFlow is a strong OJT project because it demonstrates several high-value engineering capabilities in a compact product:

- **Backend engineering:** FastAPI endpoints, typed request/response models, storage, and deployment.
- **AI integration:** foundation model gateway, prompt contracts, retrieval, and explanation generation.
- **Agent orchestration:** multi-step workflows, shared state, role-specific agents, and progress events.
- **Protocol design:** MCP tool/resource/prompt servers for modular capabilities.

- **Reliability:** schema validation, deterministic conversion, round-trip checks, tests, and audit logs.
- **Governance:** prompt-injection defenses, sensitive-data checks, human review, and risk documentation.

4.3 Target Users

User group		Value proposition
Analysts		Convert and explain files without manually writing scripts for every schema variation.
Developers		Validate API payloads, generate transformation reports, and expose repeatable workflows through APIs.
Project reviewers		See a complete AI system that combines models, agents, tools, RAG, security, and deployment.
Future teams	enterprise	Extend the same architecture to databases, drives, workflow systems, and domain-specific schema registries.

5 Proposed Solution and System Architecture

5.1 Product Concept

OJTFlow is a deployable AI platform that receives structured datasets and natural language instructions, routes work through specialized agents, retrieves trusted project knowledge, invokes deterministic tools through MCP, validates outputs, explains decisions, and asks for human confirmation when uncertainty or risk is high.

5.2 Primary Capabilities

- Natural language command interface for structured data tasks.
- Conversion among JSON, YAML, and CSV with schema preservation checks.
- Schema inference, schema validation, data profiling, and anomaly detection.
- RAG-grounded explanations using data dictionaries, transformation examples, and project documentation.
- Advanced retrieval modules: HyDE, reranking, hierarchical retrieval, GraphRAG, lightweight graph retrieval, and optional GNN reranking.
- Self-supervised representation learning for schema, field, row, table, and medical terminology embeddings.
- MCP servers for conversion, validation, retrieval, review, and audit functions.
- Multi-agent task planning with visible workflow progress.
- Human/clinician-in-the-loop checkpoints for risky, ambiguous, sensitive, or medically consequential transformations.
- Healthcare-grade explainability with evidence traces, uncertainty, limitations, and audit logs.
- API endpoints, containerized deployment, test datasets, and evaluation reports.

5.3 Non-Goals for the MVP

- No custom foundation model training in the MVP.
- No full enterprise integration in the MVP.
- No autonomous external data export without explicit human approval.
- No broad analytics platform; focus on conversion, validation, explanation, and audit.
- No multimodal document extraction unless added later as a stretch feature.

5.4 Architecture Overview

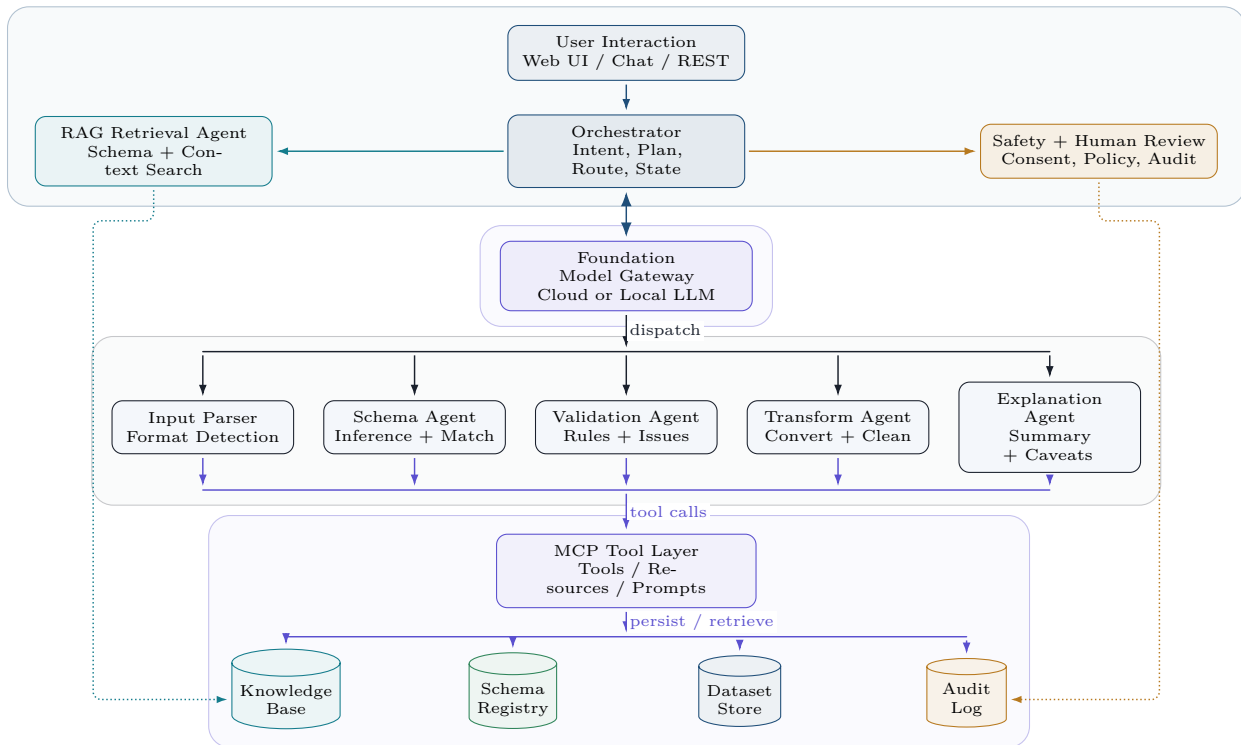


Figure 1: Architecture overview for OJTFlow. The model reasons and explains, while MCP-backed deterministic services parse, validate, transform, retrieve, review, and audit.

5.5 Layered Architecture

Layer	Responsibility	Recommended implementation
User interaction	Receives instructions, file uploads, approval decisions, and API requests.	React/Next.js or Streamlit; FastAPI REST; optional SSE.
Orchestration	Classifies intent, creates task plan, routes work to agents, manages workflow state.	LangGraph preferred; lightweight custom graph acceptable for MVP.
Agent layer	Performs parsing, schema inference, validation, transformation, retrieval, explanation, safety, and review.	Python classes or graph nodes with role-specific prompts.

Foundation model gateway	Provides natural language reasoning, planning, summarization, and explanation.	Cloud LLM or local model behind provider-neutral interface.
RAG layer	Retrieves schemas, docs, examples, rules, and error guidance.	Embeddings plus pgvector or FAISS with metadata filters.
MCP layer	Standardizes access to tools, resources, and prompts.	MCP Python SDK / FastMCP, stdio for local, Streamable HTTP for server.
Data processing	Performs exact parsing, conversion, cleaning, validation, and persistence.	Python json/csv, PyYAML, Pydantic, optional Pandas.
Storage and observability	Stores datasets, outputs, history, validation reports, approvals, logs, and traces.	SQLite for MVP; PostgreSQL + pgvector for deployment.
Deployment	Runs the stack reproducibly on development and server environments.	Docker Compose with API, MCP servers, worker, database, vector store, UI.

6 Core Components and Agent Contracts

6.1 Agent Roles

Agent	Core responsibility	Inputs	Outputs
Orchestrator	Plans workflow, chooses agents, manages state, resolves dependencies, and aggregates final response.	User request, dataset metadata, agent results.	Task plan, agent calls, response package.
Input Parser	Detects format, parses JSON/YAML/CSV, extracts field names, row counts, sample values, and parse errors.	Raw file or text.	Parsed object, format, parser report.
Schema	Infers schema, compares against known schemas, identifies required and optional fields.	Parsed data, retrieved schemas.	Schema profile, candidate match, confidence.
Validation	Checks syntax, types, missing values, duplicate keys, invalid rows, and schema violations.	Parsed data, schema profile.	Validation report, issue list, severity.
Transformation	Executes JSON/YAML/CSV conversion, field mapping, normalization, and approved cleaning.	Validated data, plan, rules.	Converted output, diff, metadata.
RAG Retrieval	Searches knowledge base for rules, examples, data dictionaries, and documentation.	User task, fields, schema candidates.	Retrieved context with source IDs.

Explanation	Produces user-facing summary of the dataset, transformation, validation findings, and caveats.	Output, validation report, retrieved context.	Explanation, glossary, limitations.
Safety and Audit	Checks prompt injection risk, sensitive data exposure, output validity, tool permissions, and logs.	Input, tool calls, outputs.	Audit log, safety flags, approval requirements.
Clinical Explainability	Builds evidence-grounded healthcare explanations, uncertainty summaries, limitations, and clinician review prompts.	Retrieved evidence, validation trace, output, clinical safety flags.	Explanation report, claim support map, review recommendation.
Human Review	Requests approval or clarification when actions are risky or ambiguous.	Proposed action, issue report.	Approval, edit, rejection, clarification.

6.2 Shared Workflow State

All agents should operate on a shared workflow state object. This prevents inconsistent interpretations and gives the system a single auditable source of truth.

```
{
  "workflow_id": "uuid",
  "user_instruction": "Clean this CSV, convert it to JSON, and explain anomalies",
  "input_format": "csv",
  "target_format": "json",
  "dataset_ref": "storage://datasets/input-001",
  "schema_profile": {},
  "retrieved_context": [],
  "validation_report": {},
  "transformation_plan": {},
  "human_approval": null,
  "output_ref": null,
  "audit_events": []
}
```

6.3 Agent Response Contract

Agent-to-agent communication should be structured rather than free-form. Each agent response should include:

Field	Purpose
status	One of success , warning , error , or needs_human_review .
summary	One short natural language description of what the agent found or changed.
data	Structured payload, never the only place where a critical decision is hidden.
confidence	Numeric or categorical confidence, especially for schema matching and repair suggestions.
issues	Errors, warnings, policy flags, or schema problems with severity.
next_recommended_action	Suggested next step for the orchestrator.

6.4 Workflow Example

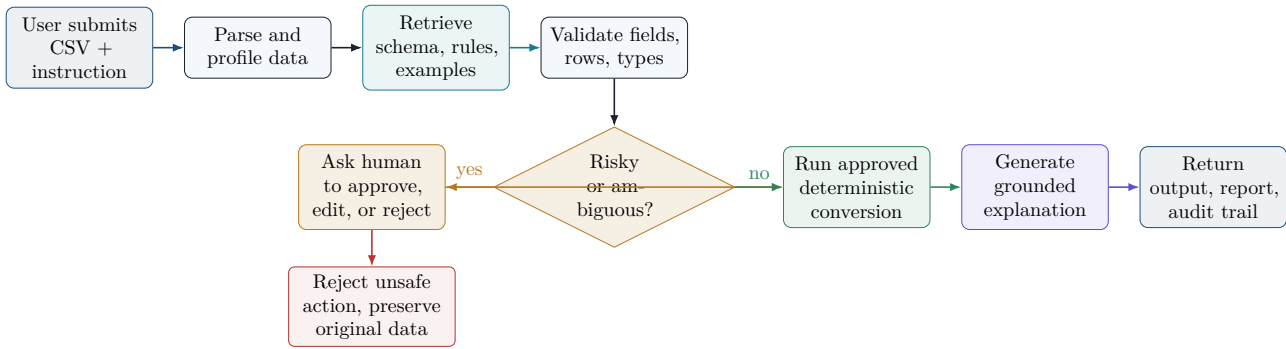


Figure 2: Human-reviewed workflow for cleaning a messy CSV, converting it to JSON, and generating a grounded explanation.

7 RAG, MCP, and Foundation Model Strategy

7.1 RAG Design

RAG should ground OJTFlow in project-specific knowledge that the model does not reliably know from training. In this proposal, retrieval is not only for answering documentation questions. It is used to improve schema matching, transformation planning, error explanation, and user-facing summaries.

7.1.1 Knowledge Sources

Source	Examples	How it helps
Schema documentation	JSON Schema files, Pydantic models, CSV column definitions.	Improves schema matching and validation explanations.
Transformation examples	Approved before/after conversions, YAML templates, CSV-to-JSON mappings.	Guides planning and explanation without inventing mapping rules.
Project documentation	Architecture notes, API docs, demo scripts, OJT requirements.	Enables project-specific help and traceable documentation.
Error library	Known parsing errors, malformed examples, repair guidance.	Improves remediation suggestions and validation messages.
Data dictionaries	Field meanings, units, business terms, enumerations.	Makes explanations useful to non-technical users.
Audit history	Past accepted transformations and approvals.	Supports consistency for repeated workflows.

7.1.2 Retrieval Pipeline

1. Ingest trusted schemas, examples, rules, and documentation.
2. Chunk content by semantic boundaries and preserve metadata: source, version, schema name, fields, trust level, and date.
3. Generate embeddings for text and schema descriptions.

4. Store vectors in pgvector for PostgreSQL deployment or FAISS for lightweight local MVP.
5. Build runtime retrieval queries from instruction, detected fields, target format, and validation issues.
6. Retrieve top-k contexts with metadata filters; optionally rerank.
7. Pass only necessary retrieved context to the model.
8. Record retrieved source IDs in the workflow trace.

7.1.3 RAG Quality Controls

- Separate trusted project knowledge from untrusted user-provided data.
- Use metadata filters to avoid mixing unrelated schemas.
- Track schema versions and retrieval source IDs.
- Require low-confidence warnings when retrieval quality is weak.
- Validate every model-suggested transformation with deterministic tools.

7.2 Advanced Retrieval Strategy

To make OJTFlow stand out technically, retrieval should be designed as a layered system rather than a single vector-search call. The MVP should ship a practical baseline, while the architecture leaves room for stronger retrieval modules when the evaluation set proves they improve relevance, faithfulness, or schema matching.

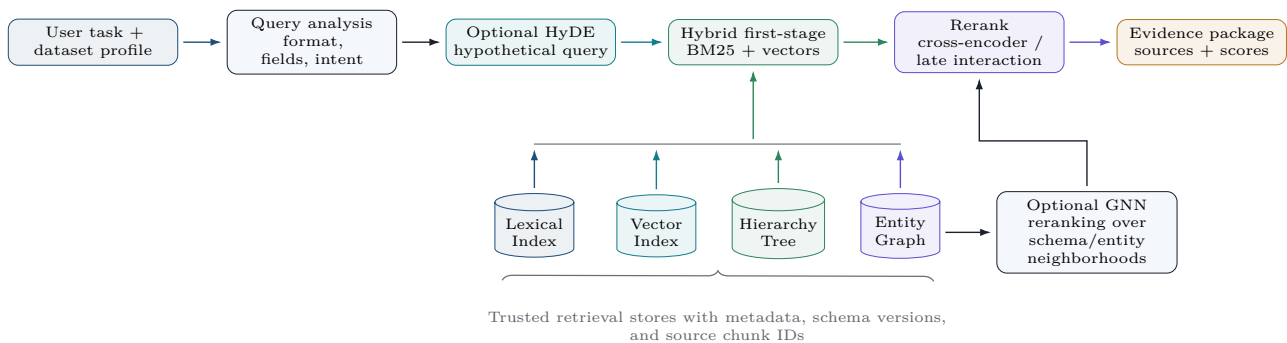


Figure 3: Advanced retrieval ladder for OJTFlow. Each level can be enabled independently and measured against the same retrieval benchmark.

7.2.1 Recommended Retrieval Ladder

Method	Role in OJTFlow	When to use	MVP status
--------	-----------------	-------------	------------

Hybrid lexical + dense retrieval	Combine BM25-style keyword matching for exact field/schema names with dense vector search for semantic matches. BEIR results support BM25 as a robust baseline and show reranking/late-interaction methods can improve zero-shot retrieval at higher cost [8].	Default retrieval for schema docs, examples, rules, and data dictionaries.	Required.
Metadata-aware filtering	Filter by source type, schema version, trust level, data format, domain, and project module before ranking.	Prevents unrelated schemas and untrusted user content from contaminating model context.	Required.
HyDE query expansion	Generate a hypothetical schema explanation, transformation note, or error report, embed it, then retrieve nearest trusted documents [3].	Useful when the user asks vague natural language questions that do not share terms with schema docs.	Optional MVP+ module.
Reranking	Rerank first-stage top-k results with a stronger relevance model. ColBERT-style late interaction offers fine-grained token-level matching [7].	Use for top 20–50 candidates where precision matters more than latency.	Demo module.
Hierarchical retrieval	Retrieve at document, section, schema, table, field, example, and chunk levels. RAPTOR-style summary trees can support long documentation and multi-step questions [4].	Use when a question needs both high-level project context and low-level field evidence.	Stretch.
Entity extraction	Extract canonical entities such as schema, field, dataset, business term, validation issue, transformation rule, agent, tool, and approval. Link aliases such as <code>customer_id</code> , <code>custId</code> , and <code>Customer ID</code> .	Essential for GraphRAG and for matching messy CSV columns to approved schemas.	Stretch.
GraphRAG	Build a graph index over entities, relationships, source chunks, and community summaries. Microsoft GraphRAG distinguishes local entity-focused search from global community-summary search [5, 6].	Use for multi-hop questions such as “which schema and cleaning rule should apply to this dataset?” or “what are the common transformation risks across sales reports?”	Stretch / re-search track.

Lightweight graph retrieval	Use a smaller graph-enhanced retrieval index when full GraphRAG indexing is too expensive. LightRAG proposes graph structures inside indexing and retrieval for simpler, faster RAG [9].	Use when the team wants graph benefits without heavy community-report generation.	Stretch.
GNN-based reranking	Apply graph neural retrieval to rank schema/entity neighborhoods for difficult multi-hop reasoning. GNN-RAG combines graph neural networks with LLM reasoning over knowledge graphs [10].	Research extension after the graph has enough labeled examples or synthetic evaluation queries.	Future re-search.

7.2.2 OJTFlow Knowledge Graph Design

The graph should be schema-guided, not open-ended. Unconstrained entity extraction often creates duplicate or noisy nodes; this project should define allowed node and edge types before ingestion.

Graph element	Examples
Node types	Dataset, Schema, Field, BusinessTerm, TransformationRule, ValidationIssue, MCPTool, Agent, ApprovalDecision, SourceChunk.
Edge types	HAS_FIELD, ALIAS_OF, VALIDATED_BY, TRANSFORMED_BY, VIOLATES, EXPLAINED_BY, APPROVED_BY, SUPPORTED_BY, DERIVED_FROM.
Retrieval modes	Local entity search for specific fields/schemas; global community search for broad summaries; hybrid graph-vector search for multi-hop schema matching.
Storage options	Start with PostgreSQL tables and NetworkX for MVP experiments; use Neo4j or a graph extension later if graph queries become central.

7.2.3 Graph-NER for Medical Entity Recognition and Schema Linking

OJTFlow should include a Graph-NER module: a hybrid clinical entity recognition, normalization, and relationship extraction layer that converts unstructured and semi-structured healthcare data into a retrievable graph. This is both a research contribution and an industrial capability because it supports de-identification, schema matching, terminology grounding, GraphRAG, and clinical audit.

Graph-NER layer	Advanced method	OJTFlow purpose
Entity recognition	Clinical NER using biomedical encoders, prompt-assisted extraction, weak supervision, and dictionary matching.	Detect patient identifiers, diagnoses, medications, procedures, labs, observations, units, dates, providers, departments, schema fields, and medical codes.
Entity normalization	Link extracted mentions to controlled concepts such as ICD, LOINC, SNOMED CT, RxNorm, FHIR resource paths, and OMOP concept IDs.	Convert messy text/CSV columns into standardized healthcare concepts for validation and retrieval.

Relationship extraction	Extract relations such as medication-dose, lab-value-unit, diagnosis-date, patient-encounter, field-schema, and rule-violation.	Build the graph used by GraphRAG, audit, data quality checks, and explanation.
Graph construction	Store nodes and edges with provenance, confidence, source chunk IDs, schema version, and PHI risk level.	Make every entity and relationship inspectable and auditable.
Graph learning	Use link prediction, node classification, and optional GNN reranking to improve entity linking and schema matching.	Research track for hard cases such as ambiguous abbreviations, site-specific columns, and multi-hop relationships.
Human correction loop	Clinicians or reviewers correct entity links and relationships; corrections become weak supervision signals.	Improves model quality while preserving accountability.

Graph-NER output contract.

```
{
  "entities": [
    {
      "text": "HbA1c",
      "type": "lab_test",
      "normalized_id": "LOINC:4548-4",
      "source_ref": "row:17,column:lab_name",
      "confidence": 0.92
    }
  ],
  "relations": [
    {
      "head": "LOINC:4548-4",
      "relation": "HAS_VALUE",
      "tail": "7.4 %",
      "evidence": "row:17",
      "confidence": 0.88
    }
  ],
  "graph_updates": ["node:lab_test", "edge:HAS_VALUE"],
  "requires_review": false
}
```

7.2.4 Safety Rules for Advanced Retrieval

- HyDE-generated text is only a retrieval query, never evidence. Final answers must cite retrieved trusted sources, validation reports, or tool outputs.
- Rerankers and GraphRAG outputs must preserve source chunk IDs so explanations remain auditable.
- Graph entity extraction should use a fixed ontology and canonicalization rules to reduce duplicate nodes.
- GraphRAG and GNN modules should be enabled only when they beat the simpler hybrid baseline on a project evaluation set.

- Retrieval should return a confidence package: retrieval mode, top-k scores, source IDs, schema versions, and missing-evidence warnings.

7.3 Self-Supervised Representation Learning Strategy

Advanced retrieval depends on representation quality. For OJTFlow, the most realistic research contribution is to improve the embedding and schema-representation layer with self-supervised learning (SSL), while using an existing foundation model for reasoning. Full foundation-model pretraining is too expensive and risky for the MVP; domain adaptation of the embedding model is feasible, measurable, and directly useful for retrieval.

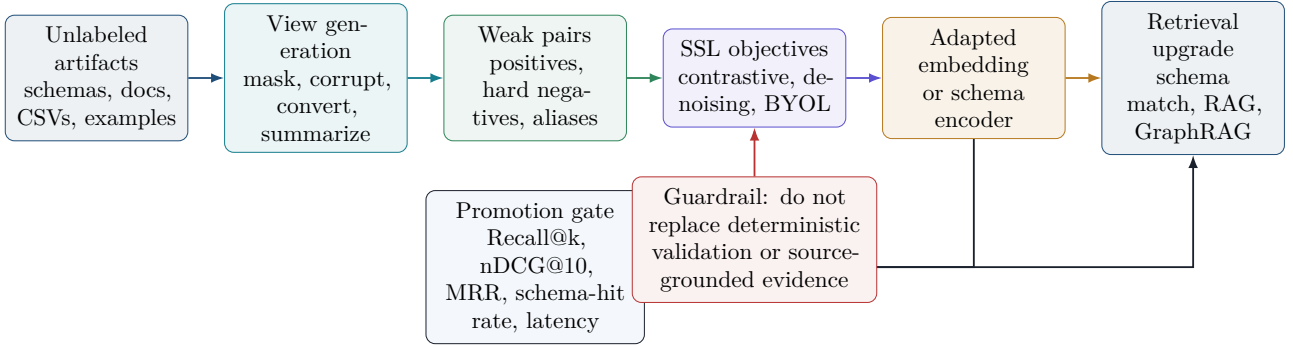


Figure 4: Self-supervised representation learning path for project-specific retrieval and schema matching.

7.3.1 Research-Informed Methods

Method family	Research basis	Use in OJTFlow
Contrastive sentence embeddings	SimCSE uses contrastive learning to improve sentence embeddings, including an unsupervised variant that creates positive pairs through dropout noise [21].	Fine-tune a bi-encoder so schema descriptions, field definitions, and user questions land near each other in embedding space.
Unsupervised dense retrieval	Contriever explores contrastive learning for unsupervised dense retrieval and shows strong transfer across retrieval settings [22].	Use as a design reference for training retrieval embeddings without manual relevance labels.
Weakly supervised embedding pretraining	E5 uses large-scale weakly supervised text pairs and contrastive learning for general-purpose embeddings [23]. OpenAI’s text/code embedding work also shows contrastive pretraining can produce strong semantic search representations [24].	Start with a strong off-the-shelf embedding model, then adapt only if project-specific retrieval tests expose gaps.
Denoising autoencoding	TSDAE learns sentence embeddings by reconstructing original text from corrupted input and is useful for domain adaptation when labeled pairs are scarce [25].	Train on project docs, schema descriptions, error messages, and transformation notes to make embeddings robust to noisy wording.

BYOL / no-negative SSL	BYOL learns representations with online and target networks without negative pairs [26]. Barlow Twins reduces redundancy between paired representations instead of relying on explicit negatives [27].	Useful when hard negatives are risky or unavailable; apply to pairs of equivalent schemas, field aliases, and format-preserving conversions.
Tabular SSL	SCARF forms contrastive tabular views through random feature corruption [28]. VIME uses value imputation and mask estimation for tabular self-supervision [29]. SubTab creates multi-view representations from feature subsets [30].	Learn row, column, and table representations for messy CSV profiling, column matching, anomaly explanation, and schema inference.

7.3.2 Project-Specific Pretext Tasks

Pretext task	Positive and negative construction	Expected benefit
Format-invariant alignment	Treat equivalent JSON, YAML, and CSV representations of the same dataset as positives; use different schemas or domains as negatives.	Embeddings learn meaning rather than file syntax.
Field alias alignment	Treat <code>customer_id</code> , <code>custId</code> , <code>Customer ID</code> , and data-dictionary descriptions as positives. Use unrelated fields as hard negatives.	Better schema matching for messy user files.
Schema-document alignment	Pair schema fields with documentation, examples, validation messages, and accepted transformations that mention them.	Better RAG retrieval for explanations and validation reports.
Transformation consistency	Pair before/after conversion examples and their approved cleaning rule. Use unsafe or rejected transformations as negatives.	Helps the retrieval layer surface relevant historical transformations.
Denoising and masking	Corrupt field names, mask column values, drop rows, or shuffle harmless ordering; train the encoder to recover stable representations.	Robustness to malformed CSVs and inconsistent naming.
Tabular view contrast	Create two views of the same table through row sampling, feature masking, random feature corruption, or column subset selection.	Stronger row/table embeddings for profiling and anomaly detection.
Graph link prediction	Predict edges such as <code>ALIAS_OF</code> , <code>HAS_FIELD</code> , <code>VALIDATED_BY</code> , and <code>DERIVED_FROM</code> .	Supports GraphRAG and optional GNN reranking.

7.3.3 Recommended Implementation Path

1. **Baseline first:** start with a strong pretrained embedding model and hybrid retrieval. Measure retrieval before training anything.
2. **Build weak labels automatically:** generate positive pairs from schemas, data dictionaries, equivalent format conversions, chunk summaries, and approved transformation examples.
3. **Fine-tune a small embedding adapter:** use contrastive loss or TSDAE-style denoising on a sentence-transformer or similar bi-encoder. Keep the base foundation model unchanged.

4. **Add tabular SSL only if needed:** use SCARF/VIME/SubTab-style augmentations for CSV/table profiling if schema inference remains weak.
5. **Try BYOL-style learning carefully:** use no-negative learning for field/schema views when reliable negatives are unavailable, with collapse checks such as embedding variance and nearest-neighbor diversity.
6. **Promote only by evaluation:** deploy the adapted embedding model only if it improves recall@k, nDCG@10, schema-hit rate, and explanation faithfulness without increasing latency beyond target.

7.3.4 Foundation Model Boundary

The proposal should not promise custom foundation-model training. A realistic path is:

- MVP: use an existing foundation model and existing embedding model.
- MVP+: adapt the embedding model or a small retrieval encoder with SSL.
- Future: use parameter-efficient fine-tuning or instruction tuning for specialized explanation behavior if enough high-quality examples exist.
- Research-only: continued pretraining of a larger foundation model on project data, considered only with compute, governance, and evaluation capacity.

7.4 MCP Integration Design

MCP gives OJTFlow a clean boundary between agent reasoning and executable capabilities. Instead of hardcoding every tool into the orchestrator, the platform can expose tool families as MCP servers.

MCP server	Resources	Tools	Prompts
structured-data-server	Dataset refs, sample rows, detected format metadata.	<code>parse_data</code> , <code>convert_json_yaml</code> , <code>convert_csv_json</code> , <code>export_file</code> .	Conversion and error templates.
schema-validation-server	Schema registry, Pydantic models, JSON Schema documents.	<code>infer_schema</code> , <code>validate_schema</code> , <code>compare_schema</code> , <code>detect_anomalies</code> .	Schema repair and validation summary prompts.
rag-context-server	Documentation chunks, field dictionaries, examples.	<code>search_context</code> , <code>get_schema_docs</code> , <code>get_examples</code> , <code>rerank_context</code> .	Grounded explanation prompts.
workflow-audit-server	Workflow logs, approval history, transformation diffs.	<code>record_event</code> , <code>fetch_history</code> , <code>generate_audit_report</code> .	Reviewer handoff and audit summary prompts.
human-review-server	Pending approval items and issue reports.	<code>request_approval</code> , <code>record_decision</code> , <code>update_plan</code> .	Clarification and approval prompts.

7.4.1 Example MCP Tool Contracts

```
tool: convert_csv_to_json
input:
  csv_text: string
  orientation: records | columns
  schema: optional object
output:
  json_text: string
  row_count: integer
  warnings: list

tool: validate_against_schema
input:
  data_ref: string
  schema_ref: string
output:
  valid: boolean
  issues: list
  severity_summary: object
```

7.5 Foundation Model Strategy

The foundation model should be treated as the reasoning and communication layer. Its responsibilities are intent interpretation, workflow planning, tool selection, explanation, summarization, and uncertainty communication. Exact parsing, conversion, schema validation, and storage should remain deterministic.

Option	Benefits	Trade-offs	Best use
Cloud LLM API	Strong reasoning, best demo quality, fast setup.	Cost, privacy review, vendor dependency.	Initial prototype and final demo.
Local open-source model	Data control, offline operation, GPU deployment.	More setup, quality variance, inference tuning.	Privacy-sensitive deployment.
Hybrid gateway	Routes simple tasks locally and complex tasks to stronger model.	More engineering complexity.	Future production-ready version.

7.6 Recommended MVP Position

Start with a model gateway abstraction and one high-quality model provider or local model. All code should call the gateway rather than hardcoding provider-specific logic. This keeps the MVP practical while preserving future flexibility.

8 Security, Governance, and Human-in-the-Loop Controls

Because OJTFlow accepts user-provided data and can invoke tools, security must be designed into the workflow rather than added at the end. The MVP should follow least privilege, typed tool input, validation after model output, and explicit approval for risky operations.

8.1 Key Risks and Mitigations

Risk	Why it matters	Mitigation
Prompt injection	User data or retrieved documents may contain hidden instructions that attempt to control the model.	Separate instructions from data, mark retrieved context as untrusted, sanitize output, use policy prompts, and validate all tool calls.
Sensitive information disclosure	Datasets may include personal, confidential, or proprietary fields.	PII detection, masking options, access controls, approval before export, and careful logging policies.
Japan medical privacy non-compliance	Japan-market pilots may involve medical history, scanned documents, or DICOM metadata that fall under APPI-sensitive handling and MHLW medical information security expectations [95, 96].	APPI-aware consent/data-use records, least privilege, tenant isolation, no raw sensitive data in logs, encryption, audit export, and review before external transfer.
Visual evidence overclaim	Segmentation masks, detection boxes, and tracking overlays can look authoritative even when they are only research outputs.	Label outputs as assistive evidence, display uncertainty, require clinician review, store source image/frame IDs, and avoid diagnostic claims without a separate SaMD path.
Improper output handling	Model outputs may be passed into tools or files without validation.	Require schema-validated JSON plans; never execute generated code; validate converted data before returning it.
Excessive agency	Agents with broad tool access can perform unintended actions.	Role-scoped tools, read-only defaults, approval gates, and deny-by-default external calls.
RAG poisoning	Untrusted documents can influence retrieved context and model behavior.	Separate trusted and untrusted indexes, use trust metadata, and reject low-confidence context.
MCP tool misuse	MCP can expose powerful tool and resource access.	Whitelist tools per agent, validate arguments with Pydantic, log calls, and require consent for write/export operations.
Schema drift	The output may follow an old or incorrect schema.	Version schemas, filter retrieval by schema version, and validate outputs against the current schema.
Cost and latency	Agentic workflows can create too many model or retrieval calls.	Cap model calls, cache schema profiles, use deterministic tools, and provide simple direct endpoints.

8.2 Human Review Triggers

Human review should be triggered whenever the system proposes an operation that can change meaning, disclose data, or affect external systems.

Trigger	Review prompt	Allowed decisions
Low schema confidence	The system found multiple possible schemas. Which one should be used?	Approve selected schema, edit, reject, clarify.
Data cleaning changes	The system proposes filling missing values, normalizing dates, or dropping malformed rows.	Approve, edit cleaning rule, reject.
Sensitive fields detected	Possible PII or confidential fields were found. Should they be masked before explanation or export?	Mask, keep with approval, reject export.
External tool/API call	The workflow wants to call an external tool or remote MCP server.	Approve or reject.
Large or costly operation	The task may exceed runtime or cost limits.	Approve, reduce scope, cancel.

8.3 Audit Requirements

Each completed workflow should store enough evidence for review:

- Workflow ID, user/session ID, timestamp, action, and tool name.
- Input hash and output hash, not unnecessary raw sensitive data in logs.
- Validation report, transformation diff, retrieved context IDs, and schema version.
- Human approval decisions, edits, or rejection reasons.
- Final status and any safety flags.

8.4 Governance Principle

The system should never hide uncertainty. If schema confidence is low, retrieved context is weak, sensitive data is present, or a cleaning action changes semantics, the user should see that explicitly and be asked to decide.

9 Medical and Healthcare Explainability Strategy

Because OJTFlow is intended for medical and healthcare data workflows, explainability must be treated as a core system requirement. The system should not merely ask a foundation model to “explain itself.” In clinical settings, natural-language rationales can be persuasive but unfaithful; recent work shows LLM self-explanations should not be trusted by default [15]. Explanation must therefore be built from verifiable evidence, validated data operations, uncertainty estimates, human review, and audit trails.

9.1 Explainability Position

OJTFlow should use a layered explainability model:

- **Data explainability:** what data was used, what format it had, what schema was inferred, what fields were missing, what values looked anomalous, and what transformations were applied.
- **Retrieval explainability:** which clinical guideline, schema, ontology entry, data dictionary, or prior validated example was retrieved; why it was considered relevant; and what evidence was not found.

- **Document and visual explainability:** which OCR page, layout block, table cell, DICOM study, image slice, frame, mask, box, or tracking segment contributed to a result.
- **Model explainability:** what prompt, retrieved evidence, tool outputs, and uncertainty signals shaped the final response. The system should not present chain-of-thought as a faithful internal explanation.
- **Workflow explainability:** which agents acted, which MCP tools were called, what validation checks passed or failed, and when human approval was required.
- **Clinical explainability:** a clinician-facing summary of intended use, limitations, uncertainty, data-quality warnings, and recommended human review points.

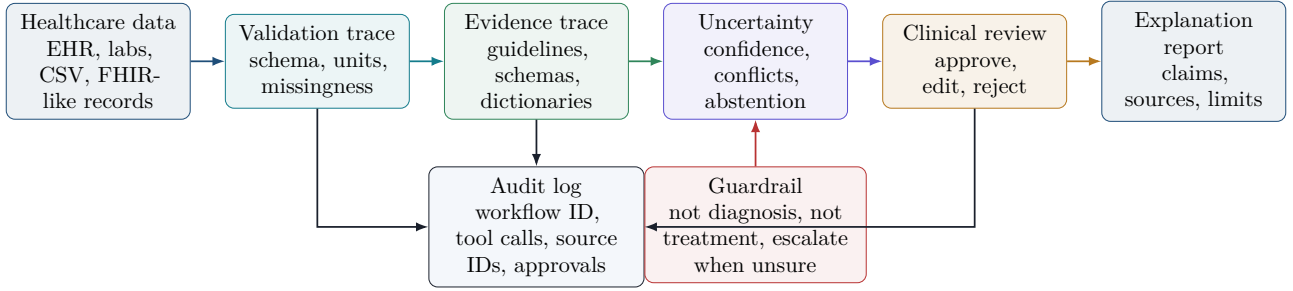


Figure 5: Healthcare-grade explainability stack for OJTFlow. Explanations are assembled from evidence, validation, uncertainty, and review traces rather than unsupported model rationales.

9.2 SOTA Explainability Methods to Apply

Method	Purpose	Use in OJTFlow
Evidence-grounded RAG explanations	Retrieval provides inspectable source evidence and provenance, reducing reliance on parametric model memory.	Every clinical or healthcare explanation should include source IDs, schema versions, retrieved context summaries, and missing-evidence warnings.
Citation checks	Generated claims should be checked against retrieved evidence and validation reports.	The Explanation Agent should mark claims as supported , partially supported , unsupported , or requires clinician review .
Counterfactual and contrastive explanations	Explain why one schema, field mapping, or transformation was chosen instead of another.	Show alternative schema candidates and the evidence that favored the selected schema.
Feature attribution	LIME, SHAP, Integrated Gradients, and TCAV provide local or concept-level explanations for model components [17, 18, 19, 20].	Use for auxiliary classifiers such as PII detection, anomaly classification, schema matching, or risk scoring. Do not overclaim them as full LLM explanations.

Attention saliency caveats	and with	Attention weights are often tempting as explanations, but NLP research warns that attention is not necessarily a faithful explanation [16].	If attention/saliency is shown, label it as a diagnostic visualization, not proof of clinical reasoning.
Mechanistic and probing methods		LLM explainability surveys categorize approaches such as probing, attribution, neuron/activation analysis, and mechanistic interpretation [14].	Future research track for model behavior auditing, not a required MVP dependency.
Uncertainty, calibration, and abstention		Medical LLM evaluation emphasizes reliability, calibration, and knowing when not to answer [13].	Require confidence scores, retrieval sufficiency checks, and explicit abstention/clinician escalation when evidence is incomplete.
Human-centered XAI evaluation		Clinical AI reporting and evaluation should include safety, human factors, workflow fit, and stakeholder-specific explanations [12, 11].	Evaluate explanations with reviewers using clarity, actionability, trust calibration, and risk-awareness rubrics.
Visual evidence explanation		Medical OCR, segmentation, detection, and tracking should be explained with source page regions, image overlays, uncertainty, and reviewer edits rather than text-only rationales.	Show OCR bounding boxes, masks, boxes, frame IDs, image metadata, and clinician correction history for every multimodal output.

9.3 Clinical Explanation Output Contract

For medical and healthcare workflows, final responses should include a structured explanation object:

```
{
  "answer_type": "data_transformation_explanation",
  "intended_use": "Support data validation and review; not autonomous diagnosis",
  "evidence": [
    {"source_id": "schema:lab_result_v2", "claim": "LOINC code is required"},
    {"source_id": "validation:wf_001", "claim": "3 rows have missing units"}
  ],
  "data_quality_flags": ["missing_units", "date_format_inconsistency"],
  "uncertainty": {
    "retrieval_confidence": "medium",
    "schema_confidence": 0.78,
    "requires_clinician_review": true
  },
  "transformation_trace": ["parse_csv", "infer_schema", "validate_schema"],
  "multimodal_evidence": {
    "ocr_boxes": ["page_2:block_7"],
    "image_regions": ["study_01:series_02:slice_18:mask_3"],
    "video_tracks": []
  },
  "unsupported_claims": [],
  "limitations": ["No diagnosis or treatment recommendation was generated"]
}
```

}

9.4 Healthcare Safety and Governance Rules

- The system should not make autonomous diagnosis, treatment, triage, or medication recommendations in the MVP.
- Explanations must separate data facts, retrieved source statements, model-generated summaries, and human decisions.
- Every clinically relevant claim should be traceable to source evidence, a validation result, or a human approval.
- Every multimodal claim should be traceable to a page/box, table cell, DICOM study/series/instance, image slice, mask, bounding box, frame, or reviewer correction.
- Low retrieval confidence, conflicting sources, missing patient context, or schema ambiguity should trigger abstention or clinician review.
- Patient-facing explanations should use plain language; clinician-facing explanations may include technical terminology, coding systems, confidence intervals, and validation details.
- Explanations should include known limitations, data gaps, and possible bias or distribution-shift warnings.

9.5 Medical Evaluation Plan

Evaluation area	Metric or test	Target
Evidence faithfulness	Percentage of explanation claims supported by retrieved sources, validation reports, or tool outputs.	$\geq 95\%$ supported for demo cases.
Clinical clarity	Reviewer rating for whether explanation is understandable and useful for the intended audience.	Average $\geq 4/5$.
Uncertainty handling	Correct abstention/escalation on incomplete, conflicting, or low-confidence evidence.	No unsafe confident answer.
Human factors	Reviewer can identify what changed, why it changed, and whether approval is required.	$\geq 90\%$ task success.
Bias and data gaps	Presence of warnings for missing demographics, skewed samples, unknown units, and schema mismatch.	All seeded gaps flagged.
Auditability	Ability to reconstruct input, retrieval, tool calls, validation, approval, and final answer.	100% for completed workflows.

10 Datasets, AI Methods, Database Stack, CI/CD, and MLOps

OJTFlow should be positioned as an advanced medical and healthcare AI platform, but the proposal must distinguish between safe prototype work and regulated clinical deployment. The recommended strategy is to use synthetic and public standards-based datasets for development, credentialed healthcare datasets for research evaluation, and protected clinical data only under formal governance.

10.1 Research and Industrial Positioning

OJTFlow should be impressive from two angles at the same time. From a research perspective, it explores advanced retrieval, Graph-NER, self-supervised representation learning, graph learning, explainable foundation AI, and medically grounded evaluation. From an industrial perspective, it is built like a real deployable platform with CI/CD, GitOps, MLOps, observability, audit trails, data governance, and clinical safety gates.

Perspective	Research contribution	Industrial deliverable
Medical data integration	FHIR/OMOP/JP Core schema matching, Graph-NER entity linking, OCR-to-structure extraction, cross-format representation learning.	Reliable conversion and validation for JSON, YAML, CSV, FHIR, Bulk FHIR, OMOP-aligned tables, scanned documents, and DICOM metadata.
Advanced retrieval	HyDE, reranking, hierarchical retrieval, GraphRAG, Graph-NER, GNN reranking.	Evidence-grounded RAG service with Qdrant/Milvus, OpenSearch, graph DB, source attribution, and retrieval SLAs.
AI modeling	Classical ML, deep tabular models, clinical NLP, medical OCR, medical computer vision, foundation AI, SSL embedding adaptation.	Versioned model registry, serving platform, evaluation gates, drift monitoring, rollback.
Explainability	Faithfulness checks, uncertainty, counterfactual explanations, attribution for auxiliary models.	Clinician-facing explanation report, audit trail, supported-claim map, intended-use statement.
Governance	Bias, robustness, privacy, calibration, external validation, reporting checklist alignment.	HIPAA-aware data handling, PHI masking, RBAC, model cards, data cards, release approvals.

10.2 Healthcare Dataset Strategy

Dataset standard	/ Why it fits OJTFlow	Primary use	Governance level
Synthea	Open-source synthetic patient generator that exports healthcare records in standards such as FHIR and CSV [42].	Safe demo data, CI tests, FHIR-to-CSV/JSON/YAML conversion, synthetic patient journeys.	Low risk; ideal for public demos.
HL7 FHIR R4	Standard for healthcare data exchange with resources such as Patient, Observation, Medication-Request, and Bundle; supports JSON and other representations [43].	Healthcare schema registry, resource validation, provenance, interoperability demos.	Standard reference.
Bulk FHIR / NDJSON	Bulk Data Access supports population-level export using structured files and is aligned with FHIR R4 [44].	Batch ingestion, population data validation, large-file conversion workflows.	Standard reference.

JP Core / Japan FHIR guidance	Japan-oriented FHIR implementation guidance for domestic patient data access and interoperability [98].	Japan-market schema registry, local profile validation, Japanese interoperability roadmap, hospital mapping demos.	Standard reference.
OMOP CDM	OHDSI OMOP standardizes observational healthcare data and vocabularies for analytics across institutions [45].	ETL target schema, clinical data warehouse mapping, vocabulary-grounded retrieval.	Standard reference.
MIMIC-IV	Deidentified EHR dataset with hospital and ICU data, available through PhysioNet credentialing [46].	Research evaluation for schema inference, table joins, ICU data quality, clinical event timelines.	Credentialed research only.
eICU-CRD	Multi-center critical care database available through PhysioNet credentialed access [47].	External validation across hospitals; robustness and distribution-shift testing.	Credentialed research only.
n2c2 / i2b2 clinical NLP	Deidentified clinical notes and shared-task datasets with data-use agreements [48].	NER, de-identification, clinical entity extraction, note-to-structured-data research.	DUA required.
Synthetic scanned forms	Generated Japanese/English referral letters, lab forms, consent documents, medication sheets, tables, and checkboxes with known ground truth.	OCR, document layout extraction, field extraction, human-review workflow, source-box provenance.	Low risk if synthetic; hospital templates require permission.
MIMIC-CXR / CheXpert	Chest X-ray datasets with reports/labels for multi-modal medical AI research [49, 50].	Imaging report retrieval, VLM explanation, report-image alignment, evidence-grounded radiology document demos.	Research only; policy dependent.
Public segmentation datasets	Medical image segmentation datasets and benchmarks used with SAM/MedSAM, nnU-Net, TotalSegmentator, and MONAI workflows [102, 105, 106, 107].	Promptable segmentation, annotation acceleration, segmentation evaluation, uncertainty overlays, evidence artifact generation.	Research only; dataset license dependent.

Public detection/tracking datasets	Public lesion, nodule, anatomy, instrument, endoscopy, ultrasound, or video datasets where licensing permits; nnDetection can provide self-configuring detection baselines [108].	Object detection/tracking proof of concept, FROC/mAP/IDF1 evaluation, model monitoring design.	Research only; no diagnostic claim.
MedQA / MedMCQA / PubMedQA	Medical exam and biomedical QA datasets for evaluating medical reasoning and evidence-grounded answering [51, 52, 53].	Foundation-model evaluation, retrieval quality, hallucination testing, answer faithfulness.	Benchmark use; not patient data.

10.3 Dataset-to-Purpose Mapping

- **CI and public demo:** Synthea + FHIR examples + handcrafted malformed CSV/JSON/YAML cases.
- **Healthcare interoperability:** FHIR R4, Bulk FHIR, OMOP CDM, JP Core/Japan FHIR guidance, and local hospital schema aliases.
- **Structured clinical research:** MIMIC-IV and eICU after credentialing and training requirements are satisfied.
- **Clinical NLP and de-identification:** n2c2/i2b2 after DUA approval.
- **Foundation AI evaluation:** MedQA, MedMCQA, PubMedQA, and curated guideline QA.
- **Clinical OCR and document AI:** synthetic Japanese/English forms, public document layouts, generated lab reports, referral letters, consent forms, and checkbox/table extraction tasks.
- **Multimodal imaging track:** MIMIC-CXR, CheXpert, public segmentation datasets, public detection/tracking datasets, DICOM samples, and report-image alignment tasks.

10.4 Medical Constraints and Requirements

Medical AI has stricter constraints than ordinary productivity AI. OJTFlow should explicitly design for these requirements:

Constraint	Why it matters	OJTFlow control
PHI and privacy	HIPAA de-identification recognizes Expert Determination and Safe Harbor methods [54].	Use synthetic data by default, PHI detectors, masking, access control, no raw PHI in logs, and approved data-use boundaries.
Japan privacy and medical information governance	APPI treats medical history as special care-required personal information, and MHLW publishes medical information system safety management guidance [95, 96].	Add Japan-market consent/data-use records, project-level tenant isolation, audit retention, encryption, access approvals, and medical information security checklist mapping.

SaMD boundary	IMDRF defines Software as a Medical Device as software intended for medical purposes without being part of hardware [55].	MVP is data support and validation, not autonomous diagnosis or treatment. Any clinical decision support scope must be explicitly classified.
Japan SaMD/program medical device boundary	PMDA provides Software as a Medical Device information and program medical device consultation pathways [97].	Japan pilots should explicitly label intended use, avoid diagnostic claims, and prepare a separate regulatory path for any clinical decision support function.
Good ML practice	FDA, Health Canada, and MHRA GMLP principles emphasize multidisciplinary expertise, representative data, independent test sets, clinically relevant performance, human factors, and monitoring [56].	Add dataset governance, external validation, subgroup analysis, HITL, monitoring, and documented model cards.
Change control	FDA PCCP guidance addresses planned modifications for AI-enabled device software functions [57].	Version prompts, models, embeddings, schemas, retrieval indexes, and evaluation gates; deploy only approved changes.
Healthcare trustworthiness	FUTURE-AI frames trustworthy healthcare AI around fairness, universality, traceability, usability, robustness, and explainability [58].	Make these six principles explicit acceptance criteria for medical workflows.
Reporting standards	TRIPOD+AI, CONSORT-AI, DECIDE-AI, and CLAIM target transparent reporting for prediction models, clinical trials, early clinical AI, and imaging AI [59, 60, 12, 61].	Use reporting checklists for evaluation reports, not just software docs.
Bias and distribution shift	Medical datasets can encode site, demographic, device, and workflow bias.	Subgroup metrics, external validation on eICU/MIMIC split, drift monitoring, and fairness reports.
Clinical accountability	Incorrect outputs can affect patient safety and clinician trust.	Human review, abstention, audit trails, intended-use labeling, and no hidden autonomous actions.

10.5 Clinical AI Acceptance Gates

Gate	Research criterion	Industrial criterion
Data readiness	Dataset is representative for the intended use, externally validated where possible, and measured for subgroup coverage.	Dataset card, license/DUA, PHI status, schema version, provenance, access control, retention policy.
Model readiness	Baselines compared against ML, DL, and foundation AI variants; calibration and uncertainty measured.	MLflow registration, reproducible pipeline, model card, approval record, rollback plan.

Retrieval readiness	Hybrid baseline compared with advanced retrieval; Graph-NER and GraphRAG improve recall or faithfulness.	Versioned vector collections, graph snapshots, source freshness checks, retrieval latency SLO.
Explainability readiness	Claims are evidence-supported; uncertainty and limitations are visible to clinicians.	Explanation template, audit trace, supported-claim map, clinician review workflow.
Safety readiness	Failure modes, bias, drift, privacy, and misuse risks are tested.	Risk register, security scan, red-team tests, monitoring dashboards, incident response plan.
Release readiness	Evaluation aligns with relevant reporting frameworks such as TRI-POD+AI, DECIDE-AI, CONSORT-AI, CLAIM, or FUTURE-AI.	CI/CD green, GitOps sync, signed image, SBOM, release notes, human approval.

10.6 AI Method Portfolio: ML, Deep Learning, and Foundation AI

OJTFlow should not be a single-model project. The strongest architecture combines classical ML, deep learning, and foundation AI, each assigned to the job it handles best.

AI layer	Methods	OJTFlow use cases
Classical machine learning	Logistic regression, gradient boosted trees, random forests, isolation forests, one-class SVM, calibrated classifiers.	Data-quality risk scoring, anomaly detection, schema candidate ranking, PII/sensitivity risk scoring, transparent baselines.
Deep learning for tables	TabNet, FT-Transformer, TabTransformer, autoencoders, contrastive tabular SSL, graph neural networks.	Column type inference, row/table embeddings, missingness patterns, schema-field matching, relational EHR graph learning.
Clinical NLP	BioClinicalBERT-style encoders, sentence transformers, NER, entity linking, de-identification models.	Medical entity extraction, PHI detection, concept normalization, note-to-structured-data research, retrieval embeddings.
Clinical OCR and document AI	OCR, layout extraction, table recognition, checkbox extraction, document VLMs, key-value extraction, and source-box provenance.	Japanese/English scanned forms, referral letters, lab reports, consent documents, and document-to-FHIR/OMOP mapping.
Medical computer vision	SAM, MedSAM, SAM 2, MedSAM-2, nnU-Net, TotalSegmentator, MONAI, nnDetection, YOLO/DETR-style detection, and video tracking.	DICOM image segmentation, lesion/object detection, medical video tracking, annotation acceleration, visual evidence overlays, and research-grade evaluation.
Graph-NER	Biomedical NER, relation extraction, concept normalization, weak supervision, graph link prediction.	Converts notes, CSV headers, FHIR paths, and OMOP fields into a clinical knowledge graph for retrieval, validation, and explanation.

Graph machine learning	Node embeddings, link prediction, GNN reranking, graph anomaly detection.	FHIR/OMOP entity graph, field aliases, transformation lineage, GraphRAG evidence navigation.
Foundation AI	LLMs, medical-tuned LLMs, embedding models, rerankers, VLMs for future imaging/report workflows.	Intent interpretation, workflow planning, explanation generation, evidence-grounded QA, multimodal future research.
Neuro-symbolic layer	Deterministic validators, JSON Schema, FHIR profiles, OMOP constraints, rule engines, ontology/vocabulary checks.	Guardrails around all ML/DL/foundation-model outputs. This is required for medical reliability.

10.7 Advanced Data and Retrieval Storage Stack

The “hardcore” version of OJTFlow should use heterogeneous stores because healthcare AI needs transactional data, vector retrieval, graph relationships, full-text search, object storage, and audit logs.

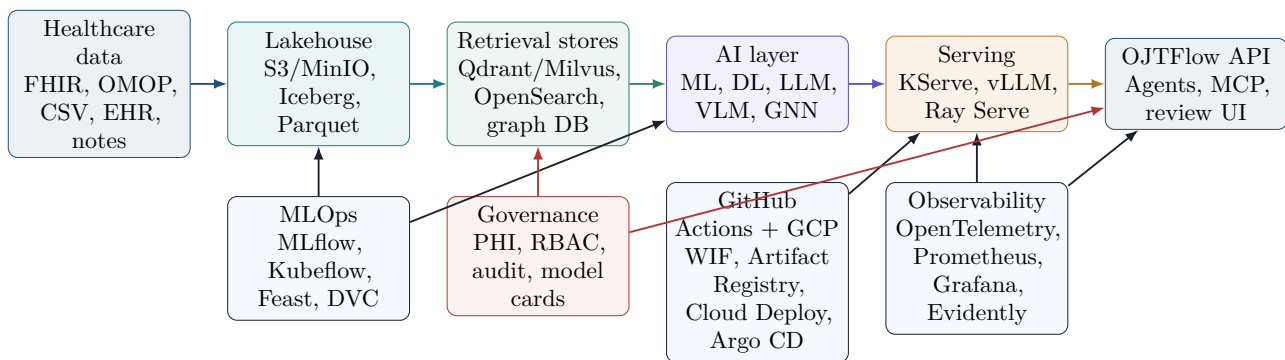


Figure 6: Advanced OJTFlow platform stack for healthcare data, AI, retrieval, MLOps, and GitOps deployment.

Store / platform	Recommended technology	Purpose
Relational source of truth	PostgreSQL	Workflows, users, permissions, approvals, schemas, audit metadata, validation reports.
Vector database	Qdrant or Milvus	Qdrant provides vector search with payload filtering and hybrid retrieval features [62]; Milvus is an open-source vector database built for GenAI scale and high-speed similarity search [63].
Hybrid search	OpenSearch / Elasticsearch plus Qdrant or Milvus	Exact medical terminology, code lookup, BM25, dense retrieval, sparse retrieval, and reranking.
Graph database	Neo4j, Memgraph, or PostgreSQL graph tables	FHIR/OMOP entity graph, schema lineage, field aliases, approval graph, GraphRAG communities.

Healthcare standard store	GCP Cloud Healthcare API	Managed FHIR, HL7v2, and DICOM stores for clinical standards, imaging metadata, de-identification workflows, and BigQuery/Vertex integration [79].
Clinical document AI	GCP Document AI Enterprise Document OCR plus optional private OCR engines	Text, layout, table, checkbox, and field extraction from scanned clinical documents with page/bounding-box provenance [100].
Lakehouse	S3/MinIO + Apache Iceberg or Delta Lake + Parquet	Versioned large datasets, raw/bronze/silver/gold zones, reproducible training and evaluation snapshots.
Feature store	Feast	Point-in-time-correct features, on-line/offline feature serving, governed context for ML and agents [64].
Model registry	MLflow	Experiment tracking, model registry, metrics, artifacts, model lineage, and promotion gates [65].
Data versioning	DVC or lakeFS	Dataset and artifact versioning, reproducible experiments, large-file governance [66].
Observability store	Prometheus, Grafana, Open-Telemetry, Evidently	Metrics, traces, logs, drift detection, data quality, RAG evaluation, LLM monitoring [67, 68, 69].

10.8 Full CI/CD, GitOps, and MLOps Pipeline

Pipeline layer	Technology	Quality gate
Source control and CI	GitHub Actions for automated workflows [70].	Lint, type checks, unit tests, integration tests, security scans, container builds.
Infrastructure as code	Terraform + Helm.	Reproducible cloud/Kubernetes infrastructure, reviewable environment changes.
Container and supply chain security	Docker/BuildKit, SBOM generation, Trivy or Gype, Cosign signing, SLSA-style provenance.	No critical vulnerabilities, signed images, dependency and license review.
GitOps CD	Argo CD, a declarative GitOps continuous delivery tool for Kubernetes [73].	Deployment only from Git, environment promotion, rollback, drift detection.
ML pipelines	Kubeflow Pipelines declare ML workflows and run them on Kubernetes [74].	Reproducible training/evaluation, cached steps, artifact lineage.

Experiment tracking	MLflow Tracking + Registry.	Promote models only after metric, fairness, calibration, latency, and safety gates pass.
Model serving	KServe for standardized Kubernetes inference [76]; vLLM for high-throughput LLM serving [77]; Ray Serve for multi-model composition [78].	Canary releases, shadow deployment, load tests, rollback, model/version pinning.
Retrieval index CI	Scheduled embedding/index builds, Qdrant/Milvus collection versioning, relevance tests.	Recall@k, nDCG@10, source coverage, stale-source detection.
MLOps monitoring	Evidently, Prometheus, Grafana, OpenTelemetry.	Data drift, embedding drift, retrieval drift, latency, cost, hallucination, unsupported-claim rate.
Clinical release gate	Human review board, model card, data card, risk report, intended-use statement.	No production promotion without documented clinical and governance approval.

10.9 GCP-First Ops and MLOps Stack

The recommended production platform for OJTFlow is Google Cloud Platform (GCP). The open-source stack remains important for portability and research reproducibility, but the industrial deployment path should be GCP-first because Google Cloud provides managed services for healthcare data, Kubernetes, CI/CD, model lifecycle, vector search, security, observability, and analytics.

GCP layer	Recommended services	OJTFlow role
Healthcare data plane	Cloud Healthcare API for FHIR, HL7v2, DICOM, and healthcare de-identification [79].	Managed healthcare ingestion and storage for clinical standards; connect FHIR stores to validation, retrieval, and audit workflows.
Document and imaging AI plane	Document AI Enterprise Document OCR, Cloud Storage, Cloud Healthcare API DICOM stores, Vertex/GKE batch jobs, and GPU node pools [100, 79].	OCR scanned Japanese/English clinical documents, store image studies, run segmentation/detection/tracking experiments, and persist visual evidence artifacts.
Data lakehouse and analytics	Cloud Storage, BigQuery, BigLake, Dataflow, Pub/Sub. BigQuery supports warehouse analytics and federated/external data access [80]; Cloud Healthcare API integrates with BigQuery and Vertex AI [79].	Store raw/bronze/silver/gold datasets, run cohort analytics, build evaluation sets, stream ingestion events, and support population-scale validation.
Kubernetes runtime	Google Kubernetes Engine (GKE), including AI/ML workloads on GKE and managed observability [81].	Run FastAPI, LangGraph workers, MCP servers, Qdrant/Milvus, graph DB, OpenSearch, KServe, Ray Serve, and GPU workloads.

Serverless runtime	Cloud Run.	Deploy lightweight APIs, webhooks, review services, and small MCP gateways when Kubernetes is unnecessary.
Build and artifact supply chain	GitHub Actions as the primary CI orchestrator, with Cloud Build as an optional GCP-native builder; Artifact Registry for images and packages; SBOM generation, vulnerability scanning, signed images. GitHub Actions supports CI/CD automation [70]; Workload Identity Federation avoids long-lived service account keys when GitHub authenticates to GCP [71, 72].	Pull-request checks, reproducible containers, model-serving images, Helm chart storage, signed release artifacts, secure GitHub-to-GCP authentication.
Continuous delivery	GitHub Actions triggers Cloud Deploy releases and Argo CD syncs Kubernetes desired state on GKE. Cloud Deploy supports delivery pipelines to GKE targets [75].	Progressive delivery to dev/staging/prod, promotion approvals, rollback, drift detection, Kubernetes sync.
ML pipelines	Vertex AI Pipelines, Kubeflow Pipelines SDK, custom training jobs [84].	Reproducible training and evaluation for ML, DL, SSL embeddings, rerankers, Graph-NER, and GNN experiments.
Model registry and serving	Vertex AI Model Registry, Vertex AI Endpoints, KServe on GKE, vLLM on GPU nodes.	Register, approve, deploy, canary, and rollback classical ML, deep learning, embedding, reranker, and foundation-model adapters.
Feature and embedding operations	Vertex AI Feature Store, Feast on GKE, Vertex AI Vector Search, Qdrant/Milvus on GKE. Vertex AI Vector Search supports RAG-capable generative AI architectures [85].	Serve point-in-time features, dense/sparse embeddings, vector indexes, schema retrieval, GraphRAG context, and medical search.
Security and privacy	IAM, Workload Identity Federation, Secret Manager, Cloud KMS, VPC Service Controls, Security Command Center. Secret Manager stores sensitive credentials [86]; VPC Service Controls reduces data exfiltration risk for services such as Cloud Storage and BigQuery [87].	PHI protection, service isolation, key management, secret rotation, least privilege, threat detection, audit readiness.
Observability and reliability	Cloud Logging, Cloud Monitoring, Cloud Trace, Managed Service for Prometheus, OpenTelemetry, Grafana, Evidently.	Distributed traces for agents and MCP calls, model latency, retrieval quality, data drift, embedding drift, unsupported-claim rate, SLO dashboards.

10.9.1 GCP Reference Deployment

1. **Landing zone:** separate GCP projects for dev, staging, production, and restricted healthcare research; enforce IAM, VPC Service Controls, Cloud KMS, audit logs, and Security Command Center.
2. **Healthcare ingestion:** store standards-based clinical data in Cloud Healthcare API FHIR, HL7v2, and DICOM stores; export analytic snapshots to BigQuery and Cloud Storage.
3. **Data platform:** use Cloud Storage + BigLake/BigQuery for lakehouse analytics; use Dataflow and Pub/Sub for streaming validation and indexing.
4. **Research platform:** run Vertex AI Pipelines for ML/DL/foundation-AI experiments; track results in MLflow or Vertex metadata; version datasets with DVC/lakeFS or Cloud Storage generation metadata.
5. **Retrieval platform:** deploy Qdrant or Milvus on GKE for controllable research experiments; use Vertex AI Vector Search as the managed production path when operational simplicity is preferred.
6. **Serving platform:** serve APIs and agents on GKE; use KServe/Ray Serve/vLLM for model endpoints; use Cloud Run for small review, webhook, and admin services.
7. **CI/CD:** use GitHub Actions as the primary CI orchestrator; authenticate to GCP through Workload Identity Federation; store images and Helm charts in Artifact Registry; promote with Cloud Deploy and Argo CD.
8. **Monitoring:** route logs, metrics, and traces to Cloud Operations; add Evidently jobs for drift and RAG quality; alert on clinical safety, security, latency, and data-quality SLOs.

10.9.2 GitHub Actions + GCP CI/CD and MLOps Blueprint

Workflow	GitHub Actions jobs	GCP integration
Pull request quality gate	Lint, type check, unit tests, API contract tests, LaTeX/documentation build, schema tests, security checks.	No GCP write access; optional read-only validation against mocked FHIR/OMOP schemas and synthetic Synthea fixtures.
Secure build	Build Docker images, generate SBOM, run vulnerability scan, sign images, publish provenance.	Authenticate with Workload Identity Federation; push signed images and Helm charts to Artifact Registry.
Data and retrieval validation	Validate FHIR/OMOP/JP Core schemas, run PHI tests, rebuild small synthetic retrieval index, run OCR fixture checks, evaluate Recall@k and nDCG@10.	Use Cloud Storage test buckets, BigQuery staging datasets, Qdrant/Milvus staging collection, Document AI test processors, or Vertex AI Vector Search staging index.
ML/DL experiment trigger	Launch classical ML, deep learning, SSL embedding, reranker, GraphNER, OCR, segmentation, detection, tracking, and GNN evaluation jobs from workflow dispatch or scheduled runs.	Trigger Vertex AI Pipelines or Kubeflow Pipelines on GKE; log metrics to MLflow/Vertex metadata; store artifacts in Cloud Storage.

Staging deploy	Deploy API, agents, MCP servers, retrieval services, model endpoints, and review UI to staging.	GitHub Actions creates Cloud Deploy release or commits a GitOps manifest change; Argo CD syncs to GKE; Cloud Run deploys lightweight services.
Clinical release gate	Require manual approval, model card, data card, risk report, retrieval report, OCR/vision report when applicable, and explainability report.	Promotion to production target in Cloud Deploy; signed image digest only; approval logs stored in PostgreSQL/BigQuery/Audit Logs.
Production monitoring	Scheduled drift tests, RAG quality tests, hallucination/unsupported-claim checks, latency and cost checks.	Cloud Monitoring, Cloud Logging, Managed Service for Prometheus, OpenTelemetry traces, Evidently reports, Security Command Center findings.

10.10 Monitorability and Observability by Design

OJTFlow must be monitorable at every layer: infrastructure, application, agent workflow, retrieval, model behavior, clinical safety, privacy, and human review. In healthcare, monitoring is not only an operations concern; it is a safety and governance requirement.

Monitoring layer	Signals to collect	GCP / stack implementation
Infrastructure	CPU, memory, GPU, disk, network, pod health, autoscaling events, queue depth.	GKE metrics, Cloud Monitoring, Managed Service for Prometheus, Grafana dashboards.
API and workflow	Request latency, error rate, workflow duration, agent step duration, MCP tool latency, retries, timeout rate.	OpenTelemetry traces, Cloud Trace, Cloud Logging, Prometheus metrics, structured workflow IDs.
Data quality	Schema violation rate, missingness, unit mismatch, malformed rows, PHI detector hits, FHIR/OMOP/JP Core validation failures, OCR extraction confidence.	Great Expectations/Pandera jobs, BigQuery quality tables, Evidently reports, Document AI metadata, alert policies.
Retrieval and RAG	Recall@k, nDCG@10, empty retrieval rate, stale source rate, source diversity, reranker score distribution, unsupported-claim rate.	Retrieval evaluation jobs, Qdrant/Milvus metrics, Vertex AI Vector Search logs, BigQuery evaluation history.
Graph-NER	Entity precision/recall samples, concept-normalization confidence, relation extraction confidence, reviewer correction rate.	Graph quality dashboards, sampled human review, graph diff reports, entity drift alerts.
Model behavior	Prediction confidence, calibration, abstention rate, hallucination flags, toxicity/safety flags, prompt injection detections, model version.	MLflow/Vertex metadata, Evidently, custom LLM evaluation jobs, OpenTelemetry span attributes.

OCR and medical vision	OCR CER/WER, field F1, table-cell F1, segmentation Dice/HD95, detection FROC/mAP, tracking IDF1/HOTA, reviewer correction rate, device/protocol drift.	Document AI logs, MLflow/Vertex metadata, GPU job metrics, DICOM metadata drift checks, sampled visual QA dashboards.
Clinical safety	Clinician override rate, unsafe-action blocks, low-confidence escalations, unsupported medical claims, delayed review items.	Clinical safety dashboard, alerting to review queue, audit log in PostgreSQL/BigQuery.
Security and privacy	Access denials, unusual data export, secret access, PHI exposure attempts, VPC Service Controls violations, vulnerability findings.	Cloud Audit Logs, Security Command Center, Secret Manager audit logs, VPC Service Controls logs.
Cost and capacity	Token cost, GPU utilization, vector DB storage, BigQuery query cost, model endpoint spend, per-workflow cost.	Cloud Billing export to BigQuery, dashboards, budget alerts, per-workflow cost tags.

10.10.1 Core SLOs and Alerts

SLO	Target	Alert trigger
Simple conversion latency	p95 under 3 seconds.	p95 exceeds target for 15 minutes.
Complex medical workflow latency	p95 under 30 seconds for demo-sized datasets.	p95 exceeds target or queue delay grows.
Validation coverage	100% of returned transformed outputs pass deterministic validation.	Any output bypasses validation.
Evidence support	$\geq 95\%$ of clinical explanation claims are source-supported in reviewed samples.	Unsupported-claim rate exceeds threshold.
PHI protection	No raw PHI in logs, prompts, or exported artifacts.	Any PHI detector hit in restricted sink.
Human review safety	100% of medically consequential or low-confidence actions require review.	Any review gate bypass event.
Retrieval health	Empty retrieval rate below threshold and source staleness within policy.	Empty/stale retrieval spikes.
OCR/vision quality	OCR field extraction, segmentation, detection, and tracking metrics stay within approved pilot thresholds.	Quality drop, device/protocol drift, or reviewer corrections exceed threshold.
Drift monitoring	Data, embedding, and retrieval drift jobs complete on schedule.	Missed job or drift severity high.

10.10.2 Runbooks and Incident Response

- Every alert should link to a runbook with owner, severity, immediate mitigation, rollback path, and evidence to collect.
- Clinical safety incidents should freeze model/index promotion, preserve audit logs, and require

human review before resuming.

- Retrieval incidents should roll back to the last known-good vector collection, graph snapshot, and prompt version.
- Model incidents should support endpoint rollback, traffic shadowing, canary disablement, and model registry demotion.
- Privacy incidents should trigger access revocation, log quarantine, PHI exposure assessment, and compliance escalation.

10.11 MLOps Lifecycle for Medical AI

1. **Data intake:** register dataset license, PHI status, de-identification method, schema, provenance, and permitted use.
2. **Data validation:** run Great Expectations or Pandera checks, FHIR/OMOP validation, PHI scans, and dataset drift baselines.
3. **Feature and embedding build:** generate tabular features, clinical entities, graph edges, dense embeddings, sparse indexes, and source metadata.
4. **Training and adaptation:** train classical ML baselines, DL/table models, SSL embedding adapters, rerankers, and optional GNN modules.
5. **Evaluation:** measure accuracy, calibration, AUROC/AUPRC where relevant, subgroup fairness, external validation, retrieval relevance, explanation faithfulness, and clinical safety tests.
6. **Registration:** store datasets, code commit, model weights, prompts, schemas, vector index version, graph snapshot, metrics, and approvals.
7. **Deployment:** deploy via GitOps to staging, canary, shadow, then production only after human approval.
8. **Monitoring:** continuously track input drift, output drift, embedding drift, retrieval quality, unsupported claims, latency, cost, security events, and human override rate.
9. **Change control:** update models, prompts, schemas, and indexes only through versioned pull requests and approved release gates.

10.12 Recommended Advanced Stack

Layer	Recommended stack
API and agents	FastAPI, Pydantic, LangGraph, MCP Python SDK / FastMCP, Celery or Ray for workers.
Healthcare standards	HL7 FHIR R4, Bulk FHIR NDJSON, OMOP CDM, JSON Schema, Pydantic models, terminology mapping.
ML/DL	scikit-learn, XGBoost/LightGBM, PyTorch, PyTorch Lightning, Hugging Face Transformers, sentence-transformers, PyTorch Geometric.
Foundation AI serving	vLLM for local/open models, cloud LLM gateway fallback, KServe/Ray Serve for composed inference.
Retrieval	Qdrant or Milvus, OpenSearch, rerankers, GraphRAG, Neo4j/Memgraph, optional GNN reranking.
Data platform	PostgreSQL, S3/MinIO, Iceberg/Delta, Parquet, DVC/lakeFS, Feast.
MLOps	MLflow, Kubeflow Pipelines, Evidently, Prometheus, Grafana, Open-Telemetry.
CI/CD and platform	GitHub Actions + GCP: Workload Identity Federation, Artifact Registry, Cloud Build optional, Cloud Deploy, GKE, Cloud Run, Terraform, Helm, Argo CD, Cosign, SBOM, vulnerability scanning.
Security and governance	OIDC/RBAC, secrets manager, audit logs, encryption, PHI masking, policy-as-code, model cards, data cards, risk registers.

11 Multimodal Medical AI and Japan Market Strategy

The full OJTFlow vision should go beyond structured tables. In Japanese healthcare, a realistic platform must handle structured records, scanned documents, referral letters, forms, radiology images, pathology images, ultrasound/endoscopy video, and the governance evidence around each output. The proposed multimodal track therefore adds OCR, document understanding, DICOM ingestion, medical image segmentation, object detection/tracking, and visual explainability to the existing RAG, Graph-NER, SSL, and MLOps architecture.

11.1 Japan-Market Rationale

Japan is an unusually strong fit for OJTFlow because its healthcare system is facing a combination of super-aging demand, medical DX policy pressure, interoperability modernization, and conservative enterprise adoption expectations. The Cabinet Office reported that people aged 65 and over represented 29.3% of Japan’s population as of October 1, 2024 [92]. MHLW’s Healthcare DX roadmap emphasizes electronic medical chart standardization, information sharing, HL7 FHIR compatibility, cloud-based standard electronic medical charts, cybersecurity, and modernization of medical facility systems [94, 93]. This creates a practical opening for a platform that can reduce data-preparation burden while staying auditable.

The Japan strategy should be careful and respectful rather than stereotyped. The proposal treats common Japanese enterprise patterns such as consensus-building, risk reduction, careful documentation, and long-term trust as product requirements. OJTFlow should support *genba* workflow fit, *ringi/nemawashi*-style approval evidence, *kaizen*-style incremental improvement, and conservative release gates. This aligns with market expectations that decisions often require internal consultation, clear supporting information, and reliability over aggressive automation [99].

11.2 Japan-Specific Compliance and Adoption Controls

Japan constraint	Implication for OJTFlow	Design response
APPI and medical data sensitivity	Japan's APPI treats medical history as special care-required personal information and restricts acquisition without consent except in defined cases [95].	Use consent-aware data flows, tenant isolation, access control, pseudonymization/de-identification options, no raw medical data in logs, and explicit data-use records.
MHLW medical information security	Healthcare information systems require governance, management, and system-operation controls [96].	Map controls to IAM, Cloud KMS, VPC Service Controls, Cloud Logging, audit retention, incident runbooks, backup/restore, and release approvals.
PMDA/SaMD boundary	Software that moves from administrative support into diagnostic or treatment decision support may enter SaMD/program medical device scope [97].	Position the prototype as assistive workflow, validation, annotation, retrieval, and explanation support; any diagnostic claim requires separate regulatory classification and clinical evaluation.
JP Core and FHIR direction	JP Core defines Japan-oriented FHIR implementation guidance, while MHLW's roadmap emphasizes standard electronic medical chart information sharing and FHIR compatibility [98, 94].	Add JP Core-aware schema registry, Japanese code-system mapping, FHIR validation profiles, and versioned mappings to OMOP/FHIR/local hospital formats.
Enterprise trust culture	Japanese hospital and enterprise buyers often need evidence, alignment, and stable operating commitments before adoption [99].	Provide reviewer queues, decision history, audit-ready reports, stakeholder sign-off, clear limitations, and measured pilot KPIs.

11.3 Multimodal Module Portfolio

Module	Advanced method	OJTFlow use case	MVP boundary
Clinical OCR and layout AI	GCP Document AI Enterprise Document OCR for text/layout extraction, plus optional local OCR engines for private deployments [100].	Extract Japanese/English text, forms, tables, checkboxes, stamps, dates, lab values, referral content, and source bounding boxes from scanned documents.	Prototype extraction and validation, not final medical record automation without review.

Document-to-graph extraction	OCR + layout-aware transformers/VLMs + Graph-NER + entity linking.	Convert scanned forms and reports into entities, relations, FHIR resources, OMOP concepts, and retrievable evidence nodes.	Human review for low-confidence fields and medically meaningful changes.
DICOM ingestion and metadata	Cloud Healthcare API supports healthcare standards including FHIR, HL7v2, and DICOM [79].	Store, search, and route radiology/pathology/endoscopy studies with provenance, modality, pixel spacing, study/series/instance IDs, and de-identification metadata.	Metadata and sample public DICOM workflow; no real patient data without governance.
Promptable segmentation	SAM, MedSAM, SAM 2, MedSAM-2, nnU-Net, TotalSegmentator, and MONAI/MONAI Label [101, 102, 103, 104, 105, 106, 107].	Clinician-assisted organ, lesion, anatomy, or region-of-interest masks; fast annotation; segmentation evidence overlays.	Research prototype and annotation assistant, not autonomous diagnosis.
Object detection	nnDetection, YOLO/RT-DETR/DETR-style detectors, Faster R-CNN baselines, and medical benchmark evaluation [108].	Detect nodules, fractures, polyps, devices, instruments, lesions, or anomalies where public datasets permit.	Measured proof of concept with public data; results labeled as research only.
Object tracking and video segmentation	SAM 2 and MedSAM-2-style tracking over ultrasound, endoscopy, laparoscopy, or 3D slice sequences [103, 104].	Track regions, instruments, or lesion boundaries across frames; support review, QA, and training data creation.	Short video/slice-stack demo with temporal consistency metrics.
Visual explainability	Masks, boxes, heatmaps, uncertainty maps, OCR bounding boxes, source pages, frame IDs, and reviewer corrections.	Make visual AI outputs inspectable: what was seen, where it was seen, how confident it was, and what a clinician corrected.	Explanation artifact generation and reviewer UI mock/API fields.

11.4 Medical Vision Research Stack

The segmentation strategy should use both foundation-style and specialist baselines. SAM provides promptable segmentation that can transfer to new image distributions [101]. MedSAM adapts the idea to medical images and was evaluated across many modalities and tasks, making it a strong research anchor for medical image segmentation [102]. SAM 2 extends promptable segmentation to images and video, while MedSAM-2 treats 2D/3D medical segmentation as a video-style tracking problem [103, 104]. nnU-Net remains a critical benchmark because it self-configures preprocessing, architecture, training, and post-processing for biomedical segmentation tasks [105]. TotalSegmentator provides a practical CT anatomy baseline for many structures [106]. MONAI Label is useful for interactive

annotation and active learning loops with clinical imaging tools [107].

- **Segmentation metrics:** Dice, IoU, Hausdorff distance/HD95, boundary F1, calibration, uncertainty, reviewer correction time, and per-modality subgroup results.
- **Detection metrics:** sensitivity, specificity, mAP, FROC, false positives per image, calibration, and clinically relevant operating points.
- **Tracking metrics:** temporal Dice/IoU, IDF1, MOTA/HOTA, identity switches, track fragmentation, latency, and frame-level reviewer correction rate.
- **OCR metrics:** character error rate, word error rate, field extraction F1, table-cell F1, checkbox accuracy, Japanese/English split performance, and human correction rate.

11.5 How Multimodal AI Connects to RAG, Graph-NER, and SSL

The multimodal track should not be separate from the existing research contribution. OCR text, layout cells, DICOM metadata, segmentation masks, detection boxes, and tracking trajectories should all become evidence objects in the same retrieval and graph architecture.

Connection	Research technique	Expected value
OCR-to-RAG	Chunk documents by page, section, table, field, and bounding box; retrieve source spans with confidence metadata.	Explanations can cite the exact page/field/box that produced a claim.
OCR-to-Graph-NER	Extract clinical entities from OCR text and link them to JP Core/FHIR resources, OMOP concepts, and hospital schemas.	Scanned Japanese documents become searchable, validated, and interoperable.
DICOM-to-RAG	Index study metadata, imaging reports, segmentation outputs, and clinician annotations.	Retrieval can combine image evidence, report evidence, and schema evidence.
Segmentation-to-explainability	Treat masks, boxes, frame IDs, and reviewer edits as evidence artifacts.	Clinicians can inspect why a region was selected and what uncertainty remains.
Self-supervised multi-modal learning	Use contrastive pairs across report-text/image, OCR-field/schema, mask/anatomy label, and graph-neighborhood views.	Better embeddings for retrieval, entity linking, and cross-modal search without requiring large labeled private datasets.
Japan localization	Embed Japanese medical terms, translated schema aliases, local coding conventions, and hospital-specific synonyms.	Higher practical adoption in Japanese hospitals and less friction during stakeholder review.

11.6 GCP-First Industrial Architecture for Multimodal Healthcare

For Japan-market deployment, the recommended operations stack should prioritize GCP regions, enterprise isolation, and traceability. Cloud Healthcare API can store standards-based FHIR, HL7v2, and DICOM data [79]; Document AI can perform OCR and layout extraction for clinical forms [100]; BigQuery and Cloud Storage support lakehouse-style analytics; and GKE/Cloud Run provide flexible

serving paths for model services. Sensitive deployment should use IAM least privilege, Workload Identity Federation from GitHub Actions, Artifact Registry, Cloud Deploy or Argo CD, Cloud KMS, Secret Manager, VPC Service Controls, Cloud Logging, Cloud Monitoring, Cloud Trace, Managed Prometheus, and audit export.

- **Data residency and tenant isolation:** keep hospital tenants isolated by project, VPC, dataset, encryption key, logging sink, and access policy.
- **Medical image pipeline:** DICOM store → de-identification/export job → segmentation/detection batch job → DICOM SEG/SR or mask artifact → evidence index.
- **Clinical document pipeline:** Cloud Storage intake → Document AI OCR → layout/entity extraction → validation/human review → FHIR/OMOP/JP Core mapping.
- **MLOps monitoring:** track OCR confidence, segmentation Dice, detection FROC, tracking consistency, model version, device/protocol drift, latency, GPU cost, and reviewer override rate.
- **Release control:** no model, prompt, schema, OCR processor, segmentation checkpoint, or retrieval index is promoted without evaluation artifacts, model/data cards, approval, rollback, and audit record.

11.7 Realistic 12-Week Prototype Scope

The multimodal scope is intentionally ambitious, but the 12-week prototype should avoid claiming clinical readiness. The deliverable should be a research-backed, Japan-ready architecture plus a small proof of concept:

- OCR proof of concept on public or synthetic Japanese/English medical-style documents with field-level confidence and human review.
- DICOM ingestion demonstration using public or synthetic data through a GCP-compatible architecture path.
- SAM/MedSAM or nnU-Net segmentation demonstration on public medical images, with masks exported as evidence artifacts.
- Detection/tracking design and one lightweight public-data experiment if time permits.
- JP Core/FHIR-aware schema registry, Japanese terminology aliases, and an APPI/MHLW/PMDA-aware governance checklist.
- Monitoring dashboard design covering structured data, RAG, Graph-NER, OCR, vision models, review actions, and cloud cost.

12 12-Week Implementation Plan and Personnel Work Breakdown

This full proposal uses a 12-week plan to show how the 4-week MVP can become a serious research and industrial prototype. The plan keeps the team assignment flexible by using unnamed member slots. Final names can be mapped to these slots later without changing the delivery structure.

12.1 Planning Assumptions

- The team has three members and keeps one weekly integration checkpoint.
- The project starts with synthetic and public healthcare standards data, synthetic/Japanese-style document samples, and public imaging examples, then prepares the governance path for credentialed datasets.

- The technical scope covers deterministic data tools, Graph-NER, advanced retrieval, self-supervised representation learning, explainable foundation AI, OCR, DICOM workflows, medical segmentation, object detection/tracking, GCP-first MLOps, CI/CD, monitoring, and auditability.
- Japan-market readiness is part of the scope: JP Core/FHIR direction, APPI/MHLW/PMDA-aware governance, Japanese terminology aliases, and approval-oriented pilot evidence.
- Clinical deployment is not claimed in 12 weeks; the goal is a medically governed prototype with evidence, risk controls, and a production-shaped cloud/MLOps path.

12.2 Member Slot Ownership

Member slot	Primary ownership	Success responsibility
Member Slot 1	Backend, architecture, deterministic tools, API contracts, workflow state, schema validation, DICOM/OCR metadata contracts, MCP tool layer, human review, and final integration.	Make OJTFlow usable as a reliable workflow platform, not only a model demo.
Member Slot 2	AI research, agent orchestration, LLM gateway, Graph-NER, self-supervised embeddings, advanced retrieval, reranking, GraphRAG context, OCR/document AI, SAM/MedSAM-style segmentation, detection/tracking design, and explanation quality.	Make the prototype impressive from a research perspective while keeping model outputs evidence-grounded.
Member Slot 3	GitHub Actions, GCP architecture, containers, MLOps, monitoring, audit logs, PHI/privacy gates, APPI/MHLW/PMDA checklist, security checks, QA, documentation, and release package.	Make the system credible from an industrial deployment and governance perspective for Japan-market pilots.

12.3 12-Week Master Schedule

Week	Theme	Main milestone	Acceptance evidence
1	Scope, governance, and architecture	Approve MVP boundaries, health-care use case, synthetic dataset strategy, module contracts, and evaluation gates.	Architecture document, backlog, risk register.
2	Core backend and data profile	Build API skeleton, parser/profiler, validation issue model, and demo data intake.	Passing parser tests and API smoke test.
3	Deterministic conversion and schema validation	Implement JSON/YAML/CSV/FHIR-like conversion, schema validation, and audit metadata.	Conversion test suite and validation report.

4	Agent workflow and MCP tool layer	Integrate orchestrator, tool contracts, workflow state, and review trigger design.	End-to-end agent workflow trace.
5	Graph-NER and medical knowledge base	Create entity schema, extractor baseline, terminology mapping, and graph storage prototype.	Entity/relation sample report.
6	Advanced retrieval baseline	Build hybrid BM25 + vector retrieval, Qdrant/Milvus collection, source metadata, HyDE experiment, and retrieval benchmark.	Recall@k, nDCG@10, source coverage.
7	Self-supervised representation learning	Generate weak pairs, run contrastive, TSDAE, SimCSE, or BYOL-style adaptation design, compare against baseline embeddings.	SSL experiment report and promotion decision.
8	Reranking, GraphRAG, and explainability	Add reranker interface, graph/hierarchy context, evidence package, supported-claim map, caveats, uncertainty, and multimodal evidence object schema.	Explanation examples and faithfulness review.
9	OCR, DICOM, and Japan-market design	Add OCR/document workflow, DICOM metadata contract, JP Core/FHIR mapping plan, Japanese terminology aliases, and APPI/MHLW/PMDA governance checklist.	OCR/DICOM design, Japan readiness checklist.
10	Medical vision and GCP MLOps integration	Prototype segmentation with SAM/MedSAM or nnU-Net, define detection/tracking experiment, build GitHub Actions/GCP deployment blueprint, and register model/evaluation artifacts.	Vision demo artifact, CI logs, registry plan.
11	Monitoring, security, and evaluation hardening	Run integrated evaluation, failure-mode testing, red-team prompts, multimodal quality checks, drift analysis plan, dashboards, and runbooks.	Evaluation report, dashboards, safety evidence.
12	Final integration and executive demo	Stabilize demo, finalize proposal package, technical report, Japan-market narrative, presentation, roadmap, and post-MVP research/production plan.	Final demo, report, slides, reproducible package.

12.4 Detailed Weekly Work Plan by Member Slot

Week	Team milestone	Member Slot 1: Backend / Architecture	Member Slot 2: AI / Retrieval / Graph-NER	Member Slot 3: GCP / MLOps / QA
1	Project foundation	Define system boundaries, module map, API surfaces, data-flow diagram, workflow states, error taxonomy, and deterministic-tool contract style. Prepare initial FastAPI/Pydantic structure and demo endpoint stubs.	Define research questions, baseline models, Graph-NER entity/relation schema, retrieval metrics, SSL embedding hypotheses, foundation-model gateway options, and clinical explanation targets.	Define GitHub branching/check policy, GCP landing-zone assumptions, PHI handling checklist, audit requirements, CI baseline, Docker convention, risk register, and weekly delivery tracker.

2	Data intake and profiling	Implement file upload/intake, JSON/YAML/CSV parser, FHIR-like bundle detector, data profiler, column/type inference, validation issue object, workflow run IDs, and basic API tests.	Build synthetic/Synthea/FHIR example corpus, initial medical dictionary, schema/document chunking rules, embedding baseline script, and first entity extraction baseline over headers and sample text.	Add lint/test GitHub Actions, Docker build skeleton, test fixture structure, dependency scanning plan, Cloud Storage/BigQuery staging design, and QA checklist for data parsing failures.
3	Validation and deterministic conversion	Implement conversion tools, schema validation path, FHIR/OMOP mapping stubs, transformation diff output, deterministic retry/error handling, audit hash fields, and unit/integration tests.	Build LLM prompt templates for intent classification and explanation, baseline retrieval query set, candidate entity normalizer, and initial comparison of lexical versus dense retrieval on schemas.	Add contract tests, container build checks, artifact naming rules, GCP Workload Identity Federation design, Artifact Registry path, and initial security scan workflow.
4	Agent orchestration and MCP contracts	Implement orchestrator state machine, parser, schema, validation, and transformation agent interfaces, MCP-compatible tool wrappers, human-review gate API, and end-to-end workflow trace.	Integrate LLM gateway with tool-only execution, design routing prompts, produce agent test cases, connect retrieval baseline to explanation agent, and define hallucination/unsupported-claim checks.	Build CI matrix for back-end/tests/docs, draft Cloud Run versus GKE deployment decision, create logging schema for workflow/agent/tool events, and define audit-log retention fields.
5	Graph-NER and medical graph	Add graph data model endpoints, entity/relation persistence hooks, schema-linking API, and backend support for graph evidence lookup.	Implement Graph-NER prototype for problems, medications, labs, procedures, PHI, FHIR paths, OMOP fields, relations, and confidence scores; connect normalized entities to graph nodes.	Add graph-store deployment options, data/versioning policy for graph snapshots, test cases for PHI/entity extraction, and dashboard requirements for entity confidence and reviewer corrections.
6	Advanced retrieval baseline	Add retrieval service interface, metadata filters, source chunk IDs, evidence package schema, retrieval trace in workflow output, and fallback behavior for empty retrieval.	Implement BM25 + vector hybrid retrieval using Qdrant/Milvus-compatible collections, HyDE query rewrite experiment, retrieval benchmark, Recall@k/nDCG/MRR metrics, and source attribution.	Add retrieval-index build workflow, collection naming/versioning rules, Cloud Storage artifact path, latency/cost measurement, and staging deployment design for vector services on GKE or managed alternatives.
7	Self-supervised representation learning	Add backend support for embedding model versions, evaluation artifacts, and feature flags for baseline versus adapted embeddings.	Generate positive/negative pairs from schemas, entity neighborhoods, validation traces, code mappings, and retrieved evidence; run contrastive, TSDAE, SimCSE-style, or BYOL-style adaptation plan; compare embedding clusters and retrieval gains.	Add MLflow/Vertex metadata plan, dataset/model card templates, experiment reproducibility checklist, GPU/runtime estimate, and promotion gate requiring measurable improvement over baseline.
8	Reranking, GraphRAG, and explanation	Integrate evidence package into final API output, add explanation report fields, caveat/uncertainty schema, multimodal evidence object schema, source-linked output sections, and review-state transitions.	Implement reranker interface, late-interaction or cross-encoder rerank option, hierarchy/tree context, GraphRAG neighborhood retrieval, optional GNN reranking design, and faithfulness/supported-claim scoring.	Add monitoring signals for retrieval relevance, empty results, stale sources, unsupported claims, prompt injection, model version, graph snapshot, and evidence artifact completeness.
9	OCR, DICOM, and Japan readiness	Add OCR document contracts, page/box provenance fields, DICOM metadata model, JP Core/FHIR mapping placeholders, Japanese terminology alias table, and review-state hooks for extracted fields.	Prototype OCR extraction on synthetic/Japanese-style forms, connect OCR text to GraphNER/RAG, define DICOM-image evidence object, and prepare Japan-market research notes.	Add APPI/MHLW/PMDA checklist, Document AI/GCP Healthcare architecture, tenant/data-residency assumptions, audit-log fields, and cloud storage policy for documents and DICOM samples.

10	Medical vision and GCP MLOps	Add mask/box/frame evidence schema, segmentation artifact storage, visual-review API fields, and integration-test fixtures for image outputs.	Run SAM/MedSAM or nnU-Net public-data segmentation demo, define detection/tracking experiment, report Dice/IoU or FROC/IDF1 plan, and connect masks/boxes to explanation artifacts.	Implement or document GitHub Actions to GCP through Workload Identity Federation, Artifact Registry, Cloud Deploy or Argo CD, GKE/Cloud Run target, Terraform/Helm skeleton, SBOM, vulnerability scan, image signing, and MLflow/Vertex/Kubeflow metadata plan.
11	Evaluation, monitoring, and hardening	Run full end-to-end tests, fix integration defects, harden failure handling, verify deterministic transformations, review OCR/image evidence paths, and prepare API/demo freeze.	Produce final metrics for Graph-NER, retrieval, SSL embeddings, OCR, segmentation/detection/tracking where available, explanation faithfulness, and foundation-model safety; document limitations and post-MVP gaps.	Build observability package: Cloud Logging, Cloud Monitoring, Cloud Trace, Managed Prometheus, Grafana/Evidently plan, PHI alerts, OCR/vision quality alerts, cost dashboard, SLOs, incident response, and release approval checklist.
12	Final demo and submission	Finalize integrated demo, architecture narrative, backend docs, module descriptions, Japan-market operating narrative, and technical handoff notes.	Finalize research report, model/retrieval/OCR/vision comparison tables, explanation examples, Graph-NER outputs, and future work for GNN reranking, multimodal medical AI, and production-scale GraphRAG.	Finalize GCP/MLOps package, runbook, audit/monitoring screenshots or mockups, APPI/MHLW/PMDA notes, cost/security notes, final slides, executive summary, and submission package.

12.5 Phase Gates and Promotion Criteria

Gate	Required evidence	Decision
MVP gate, Week 4	End-to-end workflow trace, deterministic validation, human-review trigger, and basic explanation.	Continue to research tracks only after the core workflow is stable.
Retrieval gate, Week 6	Hybrid retrieval benchmark with Recall@k, nDCG@10, MRR, source coverage, and latency.	Enable HyDE, reranker, GraphRAG, or GNN only if they improve measured quality or explainability.
Representation gate, Week 7	Baseline versus adapted embeddings on retrieval, entity linking, cluster purity, and nearest-neighbor review.	Promote SSL embedding only if it improves target metrics without collapse or drift.
Multimodal gate, Weeks 9–10	OCR confidence/field extraction report, DICOM metadata trace, segmentation artifact, visual explanation example, and Japan governance checklist.	Promote multimodal demo only as assistive review/annotation unless regulatory scope is separately approved.
Clinical safety gate, Week 11	PHI checks, unsupported-claim checks, review gates, audit logs, APPI/MHLW/PMDA notes, monitoring, and runbooks.	No final demo flow can bypass safety or governance controls.
Final readiness gate, Week 12	Reproducible package, CI evidence, evaluation report, model/data cards, OCR/vision evidence where applicable, risk register, and demo script.	Submit as a research-backed, production-shaped prototype with explicit post-MVP roadmap.

13 Technical Stack and API Design

13.1 Recommended Stack

Category	Recommended stack	Reason
Backend API	Python + FastAPI	FastAPI is a modern, high-performance Python web framework based on standard type hints [38].
Validation	Pydantic + JSON Schema	Pydantic can generate JSON Schema compliant with JSON Schema Draft 2020-12 and OpenAPI 3.1 [39].
Data conversion	Python <code>json</code> , <code>csv</code> , <code>PyYAML</code> , optional <code>Pandas</code>	Deterministic parsing and conversion for MVP formats.
Agent orchestration	LangGraph preferred; AutoGen as alternative	LangGraph provides durable execution and human-in-the-loop patterns; AutoGen is a strong research-backed multi-agent option.
RAG/vector store	pgvector for deployment; FAISS for lightweight local MVP	pgvector stores vectors inside PostgreSQL and supports exact and approximate nearest-neighbor search [40].
MCP	MCP Python SDK / FastMCP	Official SDK supports servers and clients that expose resources, prompts, tools, stdio, SSE, and Streamable HTTP [33].
Storage	SQLite for MVP; PostgreSQL for deployment	SQLite is fast to start; PostgreSQL supports durable deployment and pgvector.
Messaging/events	SSE first; Redis optional	SSE is simple for progress streams; Redis can support asynchronous workers later.
Frontend	Streamlit for fastest demo or React/Next.js for product UI	Choose based on available time and review expectations.
Deployment	Docker Compose	Compose defines and runs multi-container applications consistently from one YAML file [41].
Observability	Structured logs; OpenTelemetry later	Needed for debugging agent decisions, tool calls, and workflow state.

13.2 API Endpoints

Endpoint	Method	Purpose
/api/v1/workflows	POST	Start a multi-agent workflow with instruction, input data/file, and options.
/api/v1/workflows/{id}	GET	Fetch workflow status, events, output, explanation, and audit summary.
/api/v1/workflows/{id}/events	GET/SSE	Stream live agent progress.
/api/v1/convert	POST	Direct conversion endpoint for simple JSON/YAML/CSV tasks.
/api/v1/validate	POST	Validate data against inferred or provided schema.
/api/v1/explain	POST	Generate a grounded human-readable explanation.
/api/v1/review/{id}	POST	Submit approval, edited plan, rejection, or clarification.

13.3 Example Workflow Request

POST /api/v1/workflows

```
{
  "instruction": "Clean this CSV, convert it to JSON, and explain anomalies.",
  "input_format": "csv",
  "target_format": "json",
  "data": "date,region,sales\n2026-01-01,APAC,1000\n...",
  "options": {
    "require_human_review": true,
    "schema_name": "sales_report_v1",
    "output_orientation": "records"
  }
}
```

13.4 Example Workflow Response

```
{
  "workflow_id": "wf_001",
  "status": "needs_human_review",
  "summary": "CSV parsed successfully. Three validation issues were found.",
  "proposed_action": {
    "normalize_dates": true,
    "fill_missing_region": "Unknown",
    "drop_malformed_rows": 1
  },
  "approval_url": "/api/v1/review/wf_001"
}
```

14 Evaluation Plan and Implementation Roadmap

14.1 Evaluation Dimensions

Dimension	Metric	MVP target
Conversion correctness	Exact structural equivalence after conversion; round-trip consistency where possible.	$\geq 95\%$ on curated tests.
Validation accuracy	Correct detection of malformed rows, missing fields, type mismatches, invalid schemas.	$\geq 90\%$.
RAG relevance	Reviewer judgment of retrieved schema/docs/examples in top-k results.	$\geq 80\%$ relevant in top 5.
Retrieval ranking	Recall@k, nDCG@10, MRR, and schema-hit rate on a curated query set with expected source chunks.	Beat hybrid baseline before enabling advanced modules.
Graph retrieval utility	Correct entity linking, schema-field match accuracy, and multi-hop answer support from graph evidence.	$\geq 85\%$ correct links on demo schema set.
Graph-NER quality	Entity-level precision/recall/F1, concept-normalization accuracy, relation extraction F1, PHI recall, and reviewer correction rate.	High recall for PHI; $\geq 85\%$ F1 for demo entity types.
Representation learning	Compare pretrained embeddings against SSL-adapted embeddings using Recall@k, nDCG@10, cluster purity, schema-hit rate, and nearest-neighbor review.	Promote only with statistically meaningful gain.
OCR/document AI	Character error rate, word error rate, key-value extraction F1, table-cell F1, checkbox accuracy, bounding-box provenance, and human correction rate.	Report per document type and language; review low-confidence fields.
Segmentation	Dice, IoU, HD95, boundary F1, uncertainty quality, reviewer correction time, and modality/device subgroup checks.	Research demo only; compare against baseline.
Object detection/tracking	Detection mAP, sensitivity, FROC, false positives per image, tracking IDF1/HOTA, temporal consistency, and latency.	Research demo or design report; no diagnostic claim.
Japan-market readiness	APPI/MHLW/PMDA checklist completion, JP Core/FHIR mapping coverage, Japanese terminology alias coverage, and audit/approval evidence completeness.	Checklist complete for pilot proposal.
Explanation quality	Human rating for clarity, usefulness, and faithfulness to retrieved context and validation report.	Average $\geq 4/5$.
Workflow success	Tasks completed without incorrect routing or stalled state.	$\geq 85\%$ for MVP scenarios.
Human review effectiveness	Risky or ambiguous transformations correctly trigger review.	No critical risky action without review.
Latency	End-to-end response time for simple and complex workflows.	< 3 sec simple; < 15 sec demo workflow.

Security	Prompt injection and unsafe tool-call tests blocked or flagged.	No unapproved destructive action.
----------	---	---

14.2 Test Dataset Plan

- Valid JSON, YAML, and CSV examples for basic conversion.
- Nested JSON objects and arrays for JSON-to-YAML and YAML-to-JSON conversion.
- CSV files with missing values, malformed rows, quoted commas, numeric fields, date fields, and inconsistent columns.
- Schema cases with required fields, optional fields, enumerations, type mismatches, and duplicate keys.
- Prompt-injection strings embedded inside CSV cells and documentation chunks.
- Synthetic/Japanese-style clinical forms, referral letters, lab reports, consent forms, and checkbox/table documents for OCR.
- Public or synthetic DICOM samples with metadata, reports, segmentation masks, and source image/frame IDs.
- Public medical imaging examples for segmentation, detection, and short tracking/video demonstrations when licensing permits.
- Japan-market fixtures: JP Core/FHIR examples, Japanese terminology aliases, APPI-sensitive-field cases, and approval/audit workflows.

14.3 Roadmap Summary

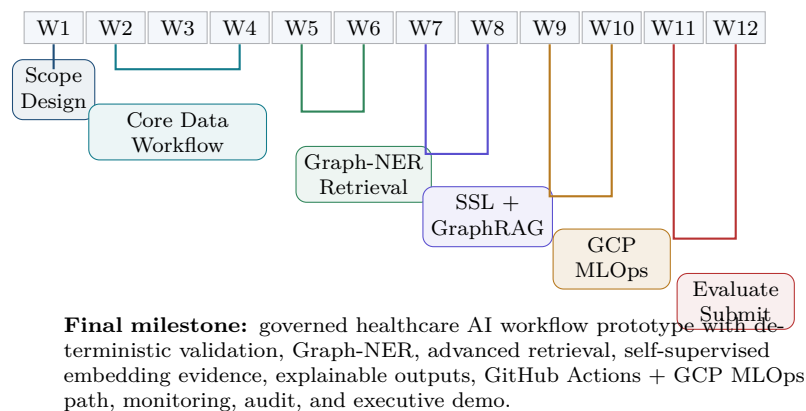


Figure 7: Twelve-week roadmap summary for the full OJTFlow prototype.

Phase	Duration	Main tasks	Deliverables
Phase 0: Scope and architecture	Week 1	Finalize medical scope, Japan-market boundary, governance boundary, datasets, module contracts, risk register, and evaluation criteria.	Approved backlog and architecture baseline.

Phase 1: Core data workflow	Weeks 2–4	Build parser, profiler, deterministic conversion, schema validation, MCP-compatible tool contracts, workflow state, and human review triggers.	Stable core MVP with traceable workflow.
Phase 2: Graph-NER and retrieval foundation	Weeks 5–6	Build medical entity extraction, normalization, graph storage, hybrid search, vector collection, HyDE experiment, and retrieval benchmark.	Graph-NER prototype and measured retrieval baseline.
Phase 3: Research acceleration	Weeks 7–8	Add self-supervised embedding adaptation, reranker interface, GraphRAG context, evidence packaging, supported-claim mapping, multimodal evidence schema, and explanation metrics.	SSL, retrieval, GraphRAG, and evidence report.
Phase 4: Japan multimodal and GCP MLOps	Weeks 9–10	Add OCR/document AI, DICOM metadata flow, SAM/MedSAM or nnU-Net segmentation proof of concept, JP Core/Japan governance checklist, GitHub Actions + GCP deployment blueprint, and model/index versioning.	Multimodal prototype and MLOps package.
Phase 5: Monitoring, evaluation, and submission	Weeks 11–12	Run integrated evaluation, multimodal quality checks, security/privacy hardening, monitoring dry run, documentation, final demo, proposal package, slides, and post-MVP roadmap.	Submission-ready research and industrial prototype.

14.4 MVP Backlog

- Backend conversion API for JSON, YAML, and CSV.
- Pydantic-based validation and schema inference.
- Agent workflow state and orchestrator.
- RAG knowledge base with schemas and transformation examples.
- Retrieval benchmark with hybrid search, reranking experiment, and source-attribution checks.
- Optional HyDE and schema-graph prototype if baseline retrieval leaves measurable gaps.
- Optional self-supervised embedding adaptation experiment using schema/document/conversion pairs.
- Graph-NER prototype for medical entity extraction, normalization, relation extraction, and schema linking.
- OCR/document AI prototype for scanned clinical-style forms with page/box provenance and human correction.
- DICOM metadata and visual-evidence object design for image/video workflows.
- SAM/MedSAM or nnU-Net segmentation proof of concept on public medical images.
- Object detection/tracking experiment design with FROC/mAP/IDF1-style evaluation plan.

- JP Core/Japan FHIR mapping notes, Japanese terminology aliases, and APPI/MHLW/PMDA governance checklist.
- MCP server for conversion and validation tools.
- Human review endpoint for ambiguous cleaning plans.
- Simple UI with input, output, explanation, and agent progress timeline.
- Docker Compose deployment.
- Test dataset suite and evaluation report.

15 Expected Outcomes, Risks, and Future Extensions

15.1 Expected Outcomes

By the end of the MVP, the team should deliver:

- A working multi-agent prototype for structured-data transformation and explanation.
- API endpoints for conversion, validation, explanation, workflow execution, and human review.
- RAG knowledge base for schemas, examples, field definitions, and project documentation.
- MCP-compatible servers for structured-data tools, validation tools, retrieval, review, and audit.
- Human-in-the-loop approval flow for risky or ambiguous transformations.
- Docker-based deployment package.
- Technical documentation, test cases, evaluation report, and final demonstration script.

15.2 Project Risks

Risk	Impact	Mitigation
Scope creep	Team may attempt too many features.	Define MVP strictly and move analytics, multimodal extraction, and enterprise integration to stretch scope.
Unreliable model outputs	Invalid structured output or incorrect explanation.	Use deterministic tools for conversion and validators after model outputs.
RAG quality issues	Wrong context may cause poor explanations.	Use curated docs, metadata filters, retrieval evaluation, and low-confidence warnings.
Retrieval complexity	HyDE, rerankers, GraphRAG, and GNN ranking can add latency, cost, and implementation risk.	Use a retrieval benchmark, ship hybrid search first, and enable advanced modules only when they outperform the baseline.
Representation learning overreach	Training an embedding or foundation model can distract from core product delivery.	Restrict MVP to pretrained embeddings; run SSL adaptation as an optional experiment with a promotion gate.
Representation collapse or drift	BYOL-style or poorly sampled contrastive learning can produce collapsed or domain-overfit embeddings.	Track embedding variance, nearest-neighbor diversity, baseline regression, and held-out schema retrieval quality.
MCP complexity	Protocol integration may consume development time.	Build direct Python tools first, then wrap stable tools as MCP servers.
GPU or model constraints	Local model performance may be insufficient.	Use model gateway; choose cloud model for demo if local inference is weak.
Security risk	Prompt injection, sensitive data leakage, or tool misuse.	Least privilege, approval gates, audit logs, adversarial tests, and sensitive-data masking.

15.3 Future Extensions

- Multi-modal data support for PDFs, images of tables, screenshots, and chart explanations.
- Domain-specific agents for finance, sales, healthcare, education, or DevOps configuration.
- GraphRAG local/global/DRIFT-style retrieval over schema, field, transformation, and approval graphs.
- GNN-based reranking for difficult multi-hop schema matching after enough labeled or synthetic training data exists.
- Self-supervised embedding adaptation using contrastive, denoising, BYOL-style, or tabular SSL objectives.
- Fine-tuned models or adapters for schema explanation and transformation planning.
- Real-time collaborative pipelines where multiple users and agents review transformations.
- Additional MCP integrations for GitHub, Google Drive, databases, dashboards, and workflow engines.
- Benchmark suite for agentic structured-data workflows.

15.4 Final Positioning

OJTFlow should be presented as a serious, deployable prototype for governed AI-assisted data operations. Its strongest claim is simple: users get the flexibility of natural language, while the system preserves the discipline of typed tools, schema validation, provenance, and human accountability.

16 References

References

- [1] R. Bommasani et al. *On the Opportunities and Risks of Foundation Models*. Stanford Center for Research on Foundation Models, 2021. <https://crfm.stanford.edu/report.html>
- [2] P. Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems 33, 2020. <https://arxiv.org/abs/2005.11401>
- [3] L. Gao, X. Ma, J. Lin, and J. Callan. *Precise Zero-Shot Dense Retrieval without Relevance Labels*. arXiv:2212.10496, 2022. <https://arxiv.org/abs/2212.10496>
- [4] P. Sarthi et al. *RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval*. arXiv:2401.18059, 2024. <https://arxiv.org/abs/2401.18059>
- [5] D. Edge et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. arXiv:2404.16130, 2024. <https://arxiv.org/abs/2404.16130>
- [6] Microsoft. *GraphRAG Documentation: Indexing Overview and Query Methods*. <https://microsoft.github.io/graphrag/index/overview/>
- [7] O. Khattab and M. Zaharia. *ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT*. SIGIR, 2020. https://people.eecs.berkeley.edu/~matei/papers/2020/sigir_colbert.pdf
- [8] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych. *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. arXiv:2104.08663, 2021. <https://arxiv.org/abs/2104.08663>
- [9] Z. Guo et al. *LightRAG: Simple and Fast Retrieval-Augmented Generation*. arXiv:2410.05779, 2024. <https://arxiv.org/abs/2410.05779>
- [10] C. Mavromatis and G. Karypis. *GNN-RAG: Graph Neural Retrieval for Large Language Model Reasoning*. arXiv:2405.20139, 2024. <https://arxiv.org/abs/2405.20139>
- [11] U.S. Food and Drug Administration, Health Canada, and Medicines and Healthcare products Regulatory Agency. *Transparency for Machine Learning-Enabled Medical Devices: Guiding Principles*. 2024. <https://www.fda.gov/medical-devices/software-medical-device-samd/transparency-machine-learning-enabled-medical-devices-guiding-principles>
- [12] B. Vasey et al. *Reporting Guideline for the Early-Stage Clinical Evaluation of Decision Support Systems Driven by Artificial Intelligence: DECIDE-AI*. Nature Medicine, 2022. <https://doi.org/10.1038/s41591-022-01772-9>
- [13] Y. Chen et al. *Evaluating Large Language Models in Medical Applications: A Survey*. arXiv:2405.07468, 2024. <https://arxiv.org/abs/2405.07468>
- [14] H. Zhao et al. *From Understanding to Utilization: A Survey on Explainability for Large Language Models*. arXiv:2401.12874, 2024. <https://arxiv.org/abs/2401.12874>

- [15] F. Huang et al. *Are Self-Explanations from Large Language Models Faithful?* Findings of ACL, 2024. <https://arxiv.org/abs/2401.07927>
- [16] S. Jain and B. C. Wallace. *Attention is not Explanation*. NAACL, 2019. <https://arxiv.org/abs/1902.10186>
- [17] M. T. Ribeiro, S. Singh, and C. Guestrin. “*Why Should I Trust You?*”: *Explaining the Predictions of Any Classifier*. KDD, 2016. <https://arxiv.org/abs/1602.04938>
- [18] S. M. Lundberg and S.-I. Lee. *A Unified Approach to Interpreting Model Predictions*. NeurIPS, 2017. <https://arxiv.org/abs/1705.07874>
- [19] M. Sundararajan, A. Taly, and Q. Yan. *Axiomatic Attribution for Deep Networks*. ICML, 2017. <https://arxiv.org/abs/1703.01365>
- [20] B. Kim et al. *Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors*. ICML, 2018. <https://arxiv.org/abs/1711.11279>
- [21] T. Gao, X. Yao, and D. Chen. *SimCSE: Simple Contrastive Learning of Sentence Embeddings*. EMNLP, 2021. <https://arxiv.org/abs/2104.08821>
- [22] G. Izacard et al. *Unsupervised Dense Information Retrieval with Contrastive Learning*. arXiv:2112.09118, 2021. <https://arxiv.org/abs/2112.09118>
- [23] L. Wang et al. *Text Embeddings by Weakly-Supervised Contrastive Pre-training*. arXiv:2212.03533, 2022. <https://arxiv.org/abs/2212.03533>
- [24] A. Neelakantan et al. *Text and Code Embeddings by Contrastive Pre-Training*. arXiv:2201.10005, 2022. <https://arxiv.org/abs/2201.10005>
- [25] K. Wang, N. Reimers, and I. Gurevych. *TSDAE: Using Transformer-based Sequential Denoising Auto-Encoder for Unsupervised Sentence Embedding Learning*. Findings of EMNLP, 2021. <https://arxiv.org/abs/2104.06979>
- [26] J.-B. Grill et al. *Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning*. NeurIPS, 2020. <https://arxiv.org/abs/2006.07733>
- [27] J. Zbontar et al. *Barlow Twins: Self-Supervised Learning via Redundancy Reduction*. ICML, 2021. <https://arxiv.org/abs/2103.03230>
- [28] D. Bahri, H. Jiang, Y. Tay, and D. Metzler. *SCARF: Self-Supervised Contrastive Learning using Random Feature Corruption*. arXiv:2106.15147, 2021. <https://arxiv.org/abs/2106.15147>
- [29] J. Yoon, Y. Zhang, J. Jordon, and M. van der Schaar. *VIME: Extending the Success of Self- and Semi-supervised Learning to Tabular Domain*. NeurIPS, 2020. <https://papers.nips.cc/paper/2020/hash/7d97667a3e056acab9aaf653807b4a03-Abstract.html>
- [30] M. P. Uddin et al. *SubTab: Subsetting Features of Tabular Data for Self-Supervised Representation Learning*. NeurIPS, 2021. <https://arxiv.org/abs/2110.04361>
- [31] Model Context Protocol. *Architecture Overview*. Official MCP documentation. <https://modelcontextprotocol.io/docs/learn/architecture>
- [32] Model Context Protocol. *Specification*. Official MCP specification. <https://modelcontextprotocol.io/specification/latest>
- [33] Model Context Protocol. *MCP Python SDK*. <https://modelcontextprotocol.github.io/python-sdk/>
- [34] Q. Wu et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. arXiv:2308.08155, 2023. <https://arxiv.org/abs/2308.08155>

- [35] LangChain. *LangGraph Documentation*. <https://docs.langchain.com/langgraph>
- [36] OWASP Foundation. *Top 10 for Large Language Model Applications*. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [37] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1, 2024. <https://doi.org/10.6028/NIST.AI.600-1>
- [38] FastAPI. *FastAPI Documentation*. <https://fastapi.tiangolo.com/>
- [39] Pydantic. *JSON Schema Documentation*. https://docs.pydantic.dev/latest/concepts/json_schema/
- [40] pgvector. *Open-source vector similarity search for Postgres*. <https://github.com/pgvector/pgvector>
- [41] Docker. *Docker Compose Documentation*. <https://docs.docker.com/compose>
- [42] The MITRE Corporation. *Synthea: Synthetic Patient Population Simulator*. <https://synthetichealth.github.io/synthea/>
- [43] HL7 International. *FHIR Release 4 Specification*. <https://hl7.org/fhir/r4/>
- [44] HL7 International. *FHIR Bulk Data Access (Flat FHIR)*. <https://hl7.org/fhir/uv/bulkdata/>
- [45] Observational Health Data Sciences and Informatics. *OMOP Common Data Model*. <https://ohdsi.github.io/CommonDataModel/>
- [46] PhysioNet. *MIMIC-IV*. <https://physionet.org/content/mimiciv/>
- [47] PhysioNet. *eICU Collaborative Research Database*. <https://physionet.org/content/eicu-crd/2.0/>
- [48] National NLP Clinical Challenges. *n2c2 / i2b2 Clinical NLP Data Sets*. <https://www.i2b2.org/NLP/DataSets/>
- [49] A. E. W. Johnson et al. *MIMIC-CXR, a de-identified publicly available database of chest radiographs with free-text reports*. Scientific Data, 2019. <https://www.nature.com/articles/s41597-019-0322-0>
- [50] Stanford AIMI. *CheXpert: Chest X-rays*. <https://aimi.stanford.edu/datasets/chexpert-chest-x-rays>
- [51] D. Jin et al. *What Disease does this Patient Have? A Large-scale Open Domain Question Answering Dataset from Medical Exams*. arXiv:2009.13081, 2020. <https://arxiv.org/abs/2009.13081>
- [52] A. Pal, L. Umapathi, and M. Sankarasubbu. *MedMCQA: A Large-scale Multi-Subject Multi-Choice Dataset for Medical Domain Question Answering*. arXiv:2203.14371, 2022. <https://arxiv.org/abs/2203.14371>
- [53] Q. Jin et al. *PubMedQA: A Dataset for Biomedical Research Question Answering*. EMNLP, 2019. <https://arxiv.org/abs/1909.06146>
- [54] U.S. Department of Health and Human Services. *Methods for De-identification of Protected Health Information*. <https://www.hhs.gov/hipaa/for-professionals/privacy/special-topics/de-identification/>
- [55] International Medical Device Regulators Forum. *Software as a Medical Device: Key Definitions*. <https://www.imdrf.org/documents/software-medical-device-samd-key-definitions>

- [56] U.S. Food and Drug Administration. *Good Machine Learning Practice for Medical Device Development: Guiding Principles*. <https://www.fda.gov/medical-devices/software-medical-device-samd/good-machine-learning-practice-medical-device-development-guiding-principles>
- [57] U.S. Food and Drug Administration. *Marketing Submission Recommendations for a Predetermined Change Control Plan for Artificial Intelligence-Enabled Device Software Functions*. FDA guidance, 2025.
- [58] K. Lekadir et al. *FUTURE-AI: International Consensus Guideline for Trustworthy and Deployable Artificial Intelligence in Healthcare*. BMJ, 2025. <https://www.bmj.com/content/388/bmj-2024-081554>
- [59] G. S. Collins et al. *TRIPOD+AI Statement: Updated Guidance for Reporting Clinical Prediction Models*. BMJ, 2024. <https://www.bmj.com/content/385/bmj.q902>
- [60] X. Liu et al. *Reporting Guidelines for Clinical Trial Reports for Interventions Involving Artificial Intelligence: The CONSORT-AI Extension*. Nature Medicine, 2020. <https://www.nature.com/articles/s41591-020-1034-x>
- [61] J. Mongan et al. *Checklist for Artificial Intelligence in Medical Imaging (CLAIM)*. Radiology: Artificial Intelligence, 2020. <https://pubs.rsna.org/doi/10.1148/ryai.2020200029>
- [62] Qdrant. *Qdrant Vector Search Engine Documentation*. <https://qdrant.tech/documentation/overview/>
- [63] Milvus. *High-Performance Vector Database Built for Scale*. <https://milvus.io/>
- [64] Feast. *The Open Source Feature Store for Machine Learning*. <https://docs.feast.dev/>
- [65] MLflow. *MLflow Model Registry*. <https://mlflow.org/docs/latest/ml/model-registry/>
- [66] Data Version Control. *Versioning Data and Models*. <https://dvc.org/doc/use-cases/versioning-data-and-models>
- [67] OpenTelemetry. *OpenTelemetry Documentation*. <https://opentelemetry.io/docs/>
- [68] Prometheus. *Overview*. <https://prometheus.io/docs/introduction/overview/>
- [69] Evidently AI. *Open-Source ML Monitoring and LLM Observability*. <https://www.evidentlyai.com/evidently-oss>
- [70] GitHub. *GitHub Actions Documentation*. <https://docs.github.com/en/actions>
- [71] Google Cloud. *Workload Identity Federation*. <https://cloud.google.com/iam/docs/workload-identity-federation>
- [72] GitHub. *Configuring OpenID Connect in Google Cloud Platform*. <https://docs.github.com/en/actions/security-for-github-actions/security-hardening-your-deployments/configuring-openid-connect-in-google-cloud-platform>
- [73] Argo CD. *Declarative GitOps CD for Kubernetes*. <https://argo-cd.readthedocs.io/>
- [74] KubeFlow. *KubeFlow Pipelines*. <https://www.kubeflow.org/docs/components/pipelines/concepts/pipeline/>
- [75] Google Cloud. *Deploy an App to GKE using Cloud Deploy*. <https://cloud.google.com/deploy/docs/deploy-app-gke>
- [76] KServe. *Standardized Distributed Inference Platform on Kubernetes*. <https://kserve.github.io/kserve/>

- [77] vLLM. *High-Throughput and Memory-Efficient LLM Serving*. <https://www.vllm.ai/>
- [78] Ray. *Ray Serve: Scalable and Programmable Serving*. <https://docs.ray.io/en/latest/serve/>
- [79] Google Cloud. *Cloud Healthcare API*. <https://cloud.google.com/healthcare-api>
- [80] Google Cloud. *BigQuery Overview*. <https://docs.cloud.google.com/bigquery/docs/introduction>
- [81] Google Cloud. *Google Kubernetes Engine Documentation*. <https://docs.cloud.google.com/kubernetes-engine/docs/about>
- [82] Google Cloud. *Cloud Build Documentation*. <https://cloud.google.com/build/docs>
- [83] Google Cloud. *Artifact Registry Supported Formats*. <https://docs.cloud.google.com/artifact-registry/docs/supported-formats>
- [84] Google Cloud. *Introduction to Vertex AI Pipelines*. <https://cloud.google.com/vertex-ai/docs/pipelines/introduction>
- [85] Google Cloud. *Vertex AI Vector Search*. <https://docs.cloud.google.com/vertex-ai/docs/vector-search/overview>
- [86] Google Cloud. *Secret Manager Overview*. <https://docs.cloud.google.com/secret-manager/docs/overview>
- [87] Google Cloud. *Overview of VPC Service Controls*. <https://docs.cloud.google.com/vpc-service-controls/docs/overview>
- [88] U.S. National Library of Medicine. *Unified Medical Language System*. <https://www.nlm.nih.gov/research/umls/index.html>
- [89] SNOMED International. *SNOMED CT*. <https://www.snomed.org/snomed-ct>
- [90] Regenstrief Institute. *LOINC*. <https://loinc.org/>
- [91] U.S. National Library of Medicine. *RxNorm*. <https://www.nlm.nih.gov/research/umls/rxnorm/index.html>
- [92] Cabinet Office, Government of Japan. *Annual Report on the Ageing Society 2025*. <https://www8.cao.go.jp/kourei/english/annualreport/index-wh.html>
- [93] Ministry of Health, Labour and Welfare, Japan. *Medical DX*. <https://www.mhlw.go.jp/stf/iryoudx.html>
- [94] Ministry of Health, Labour and Welfare, Japan. *Roadmap for the Promotion of Healthcare DX*. <https://www.mhlw.go.jp/english/policy/health-medical/healthcare-dx/dl/Roadmap-for-the-Promotion-of-Healthcare-DX%28Digital-Transformation%29.pdf>
- [95] Personal Information Protection Commission, Japan. *Act on the Protection of Personal Information*. https://www.ppc.go.jp/files/pdf/APPI_english.pdf
- [96] Ministry of Health, Labour and Welfare, Japan. *Guidelines on Safety Management of Healthcare Information Systems Version 6.0*. https://www.mhlw.go.jp/stf/shingi/0000516275_00006.html
- [97] Pharmaceuticals and Medical Devices Agency. *Software as a Medical Device*. <https://www.pmda.go.jp/english/review-services/reviews/0009.html>
- [98] FHIR Japanese Implementation Research Working Group. *HL7 FHIR JP Core Implementation Guide*. <https://www.jpfhir.jp/fhir/core/latest/guide-general.html>

- [99] Global Business Culture. *Japan Business Culture Guide: Communication, Leadership, and Decision-Making*. <https://www.globalbusinessculture.com/countries/japan/japan-business-culture/>
- [100] Google Cloud. *Enterprise Document OCR*. <https://docs.cloud.google.com/document-ai/docs/enterprise-document-ocr>
- [101] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollar, and Ross Girshick. *Segment Anything*. arXiv:2304.02643, 2023. <https://arxiv.org/abs/2304.02643>
- [102] Jun Ma, Yuting He, Feifei Li, Lin Han, Chenyu You, and Bo Wang. *Segment Anything in Medical Images*. Nature Communications, 15:654, 2024. <https://www.nature.com/articles/s41467-024-44824-z>
- [103] Nikhila Ravi et al. *SAM 2: Segment Anything in Images and Videos*. arXiv:2408.00714, 2024. <https://arxiv.org/abs/2408.00714>
- [104] Junde Wu et al. *Medical SAM 2: Segment Medical Images as Video via Segment Anything Model 2*. arXiv:2408.00874, 2024. <https://arxiv.org/abs/2408.00874>
- [105] Fabian Isensee, Paul F. Jaeger, Simon A. A. Kohl, Jens Petersen, and Klaus H. Maier-Hein. *nnU-Net: A Self-Configuring Method for Deep Learning-Based Biomedical Image Segmentation*. Nature Methods, 18:203–211, 2021. <https://www.nature.com/articles/s41592-020-01008-z>
- [106] Jakob Wasserthal et al. *TotalSegmentator: Robust Segmentation of 104 Anatomical Structures in CT Images*. arXiv:2208.05868, 2022. <https://arxiv.org/abs/2208.05868>
- [107] MONAI Consortium. *MONAI Label Documentation*. <https://docs.monai.io/projects/label>
- [108] Michael Baumgartner, Paul F. Jaeger, Fabian Isensee, and Klaus H. Maier-Hein. *nnDetection: A Self-Configuring Method for Medical Object Detection*. arXiv:2106.00817, 2021. <https://arxiv.org/abs/2106.00817>